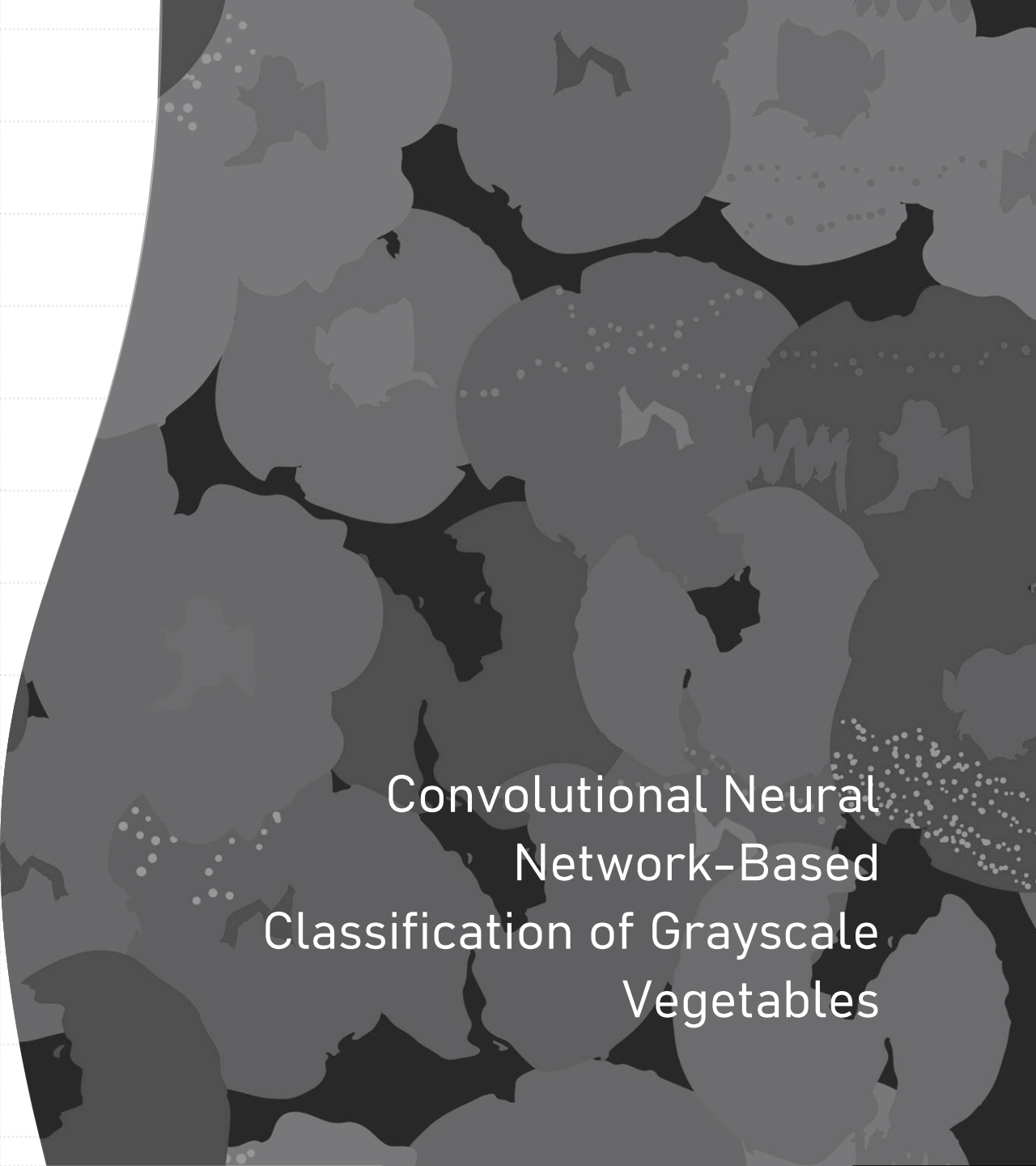




Part A – CNN

By - Ng Yu Hang
DAAA/FT/2B/23
P2423500



Convolutional Neural
Network-Based
Classification of Grayscale
Vegetables

Project information + Goals

Project Information:

- Build multiple CNN models, comparing different configurations and architectures to make the best model for classification of 11 classes of vegetables (shown next slide) for resolutions **23x23** & **101x101**

Goals:

- Make a **experiment + report-style** notebook that:
 - Is **reproducible** (consistent results with the use of set seeds (42))
 - **Modular** (pre-defined functions)
 - Is very **scalable** (Multi-input-size compatible)
 - Clean and structured experimental design phases.
 - Has **clean and clear documentation** with **explainable** models
 - Follows **industry-standard practices** (informative model naming + centralized data augmentation pipelines + models could adjust to input size)

```
SEED = 42  
  
np.random.seed(SEED)  
random.seed(SEED)  
tf.random.set_seed(SEED)  
os.environ['PYTHONHASHSEED'] = str(SEED)
```

Our dataset – 11 classes of vegetables



Beans



Bitter Gourd



Brinjal



Cabbage



Capsicum



Cauliflower
& Broccoli



Cucumber &
Bottle Gourd



Potato



Pumpkin

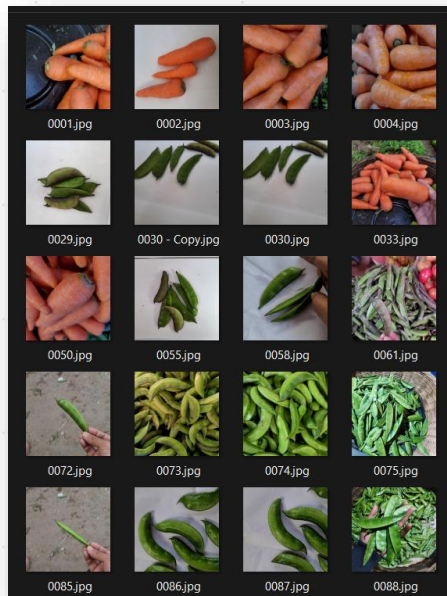


Radish &
Carrot



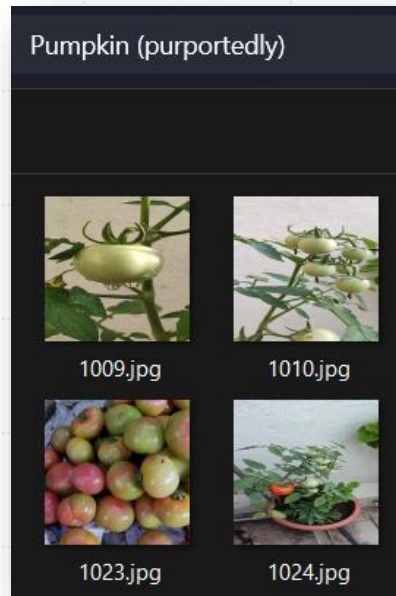
Tomato

Identified issues before project



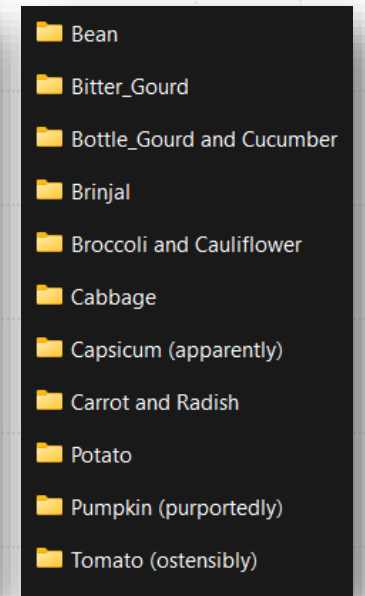
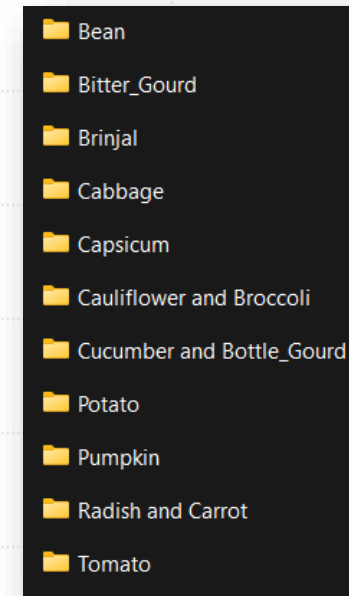
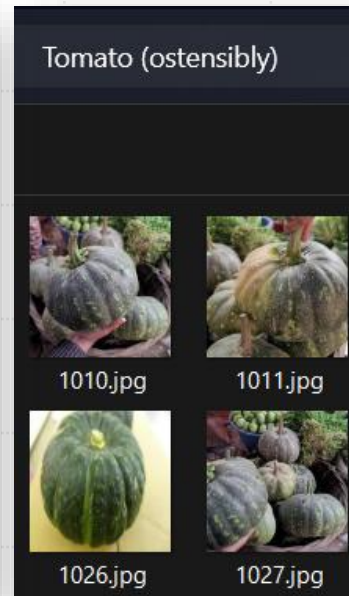
Train data (Beans)
contains carrots

Manual deletion



Test Data – Pumpkin and Tomato
swapped

Manual Swapping

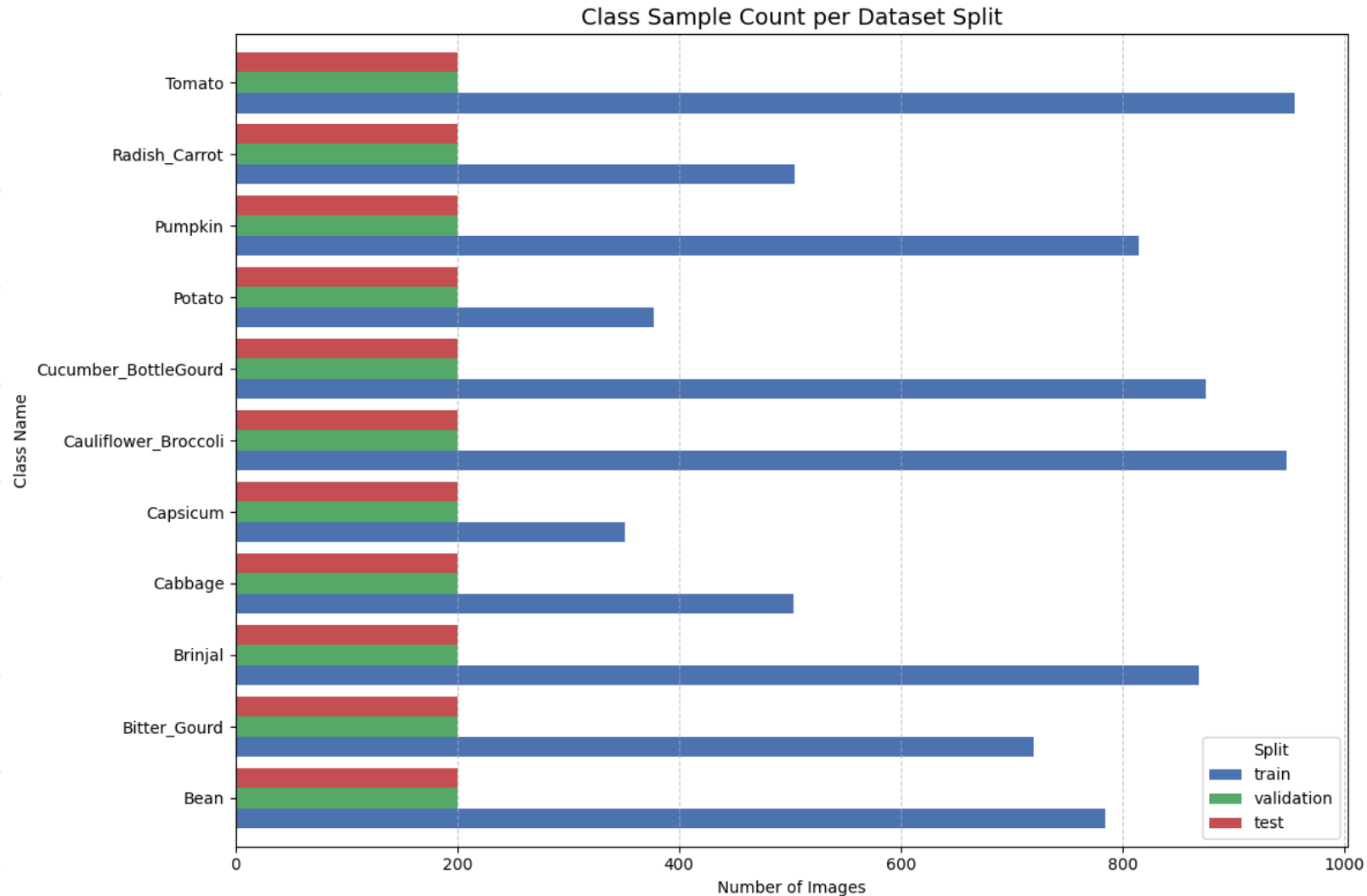


Inconsistent Naming across
train/val/test datasets

**This causes it to be treated
as a separate class if not fixed!**

Fixed by file name
standardization in code

Issues identified in the project



Imbalanced data!

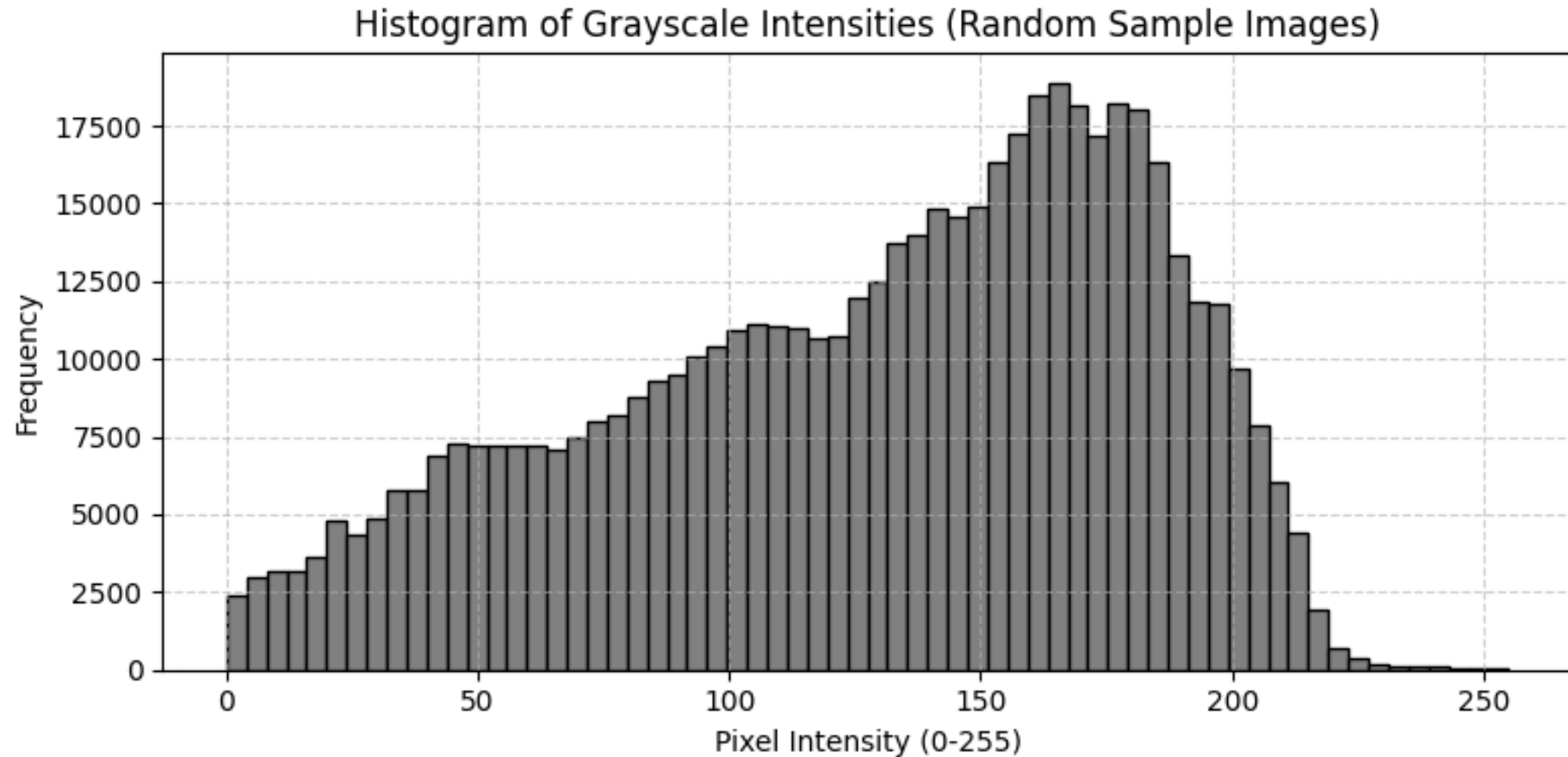
Over/under
representation

BIAS!!

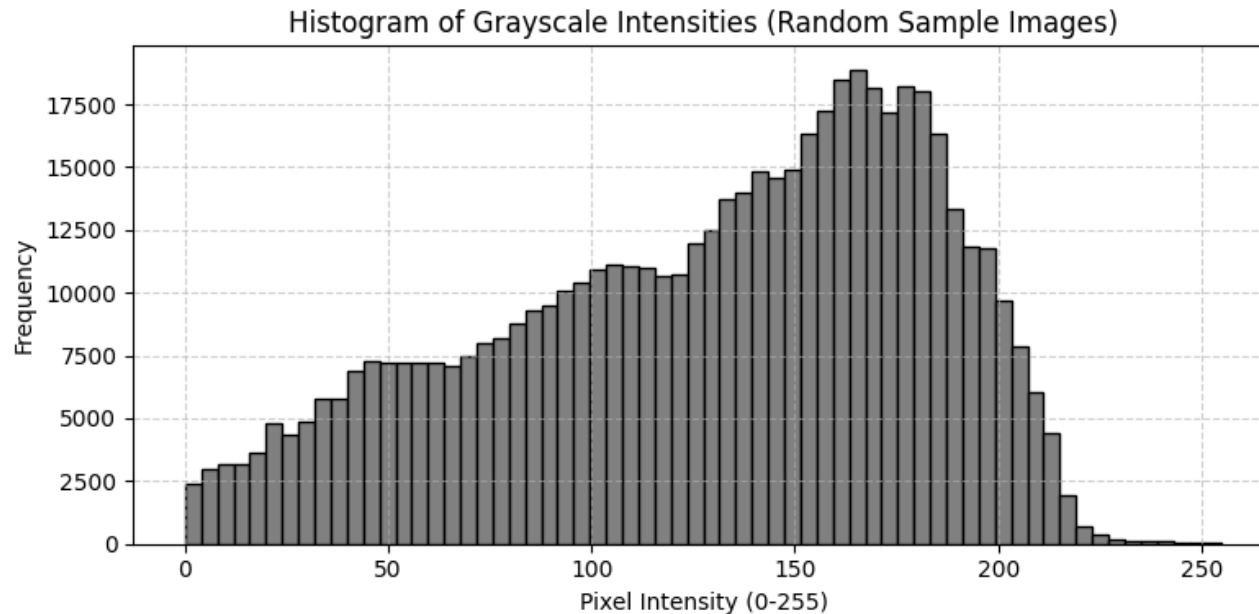
	train	validation	test
Bean	784	200	200
Bitter_Gourd	720	200	200
Brinjal	868	200	200
Cabbage	503	200	200
Capsicum	351	200	200
Cauliflower_Broccoli	948	200	200
Cucumber_BottleGourd	875	200	200
Potato	377	200	200
Pumpkin	814	200	200
Radish_Carrot	504	200	200
Tomato	955	200	200

Solved by assigning class
weights

Exploratory Data Analysis



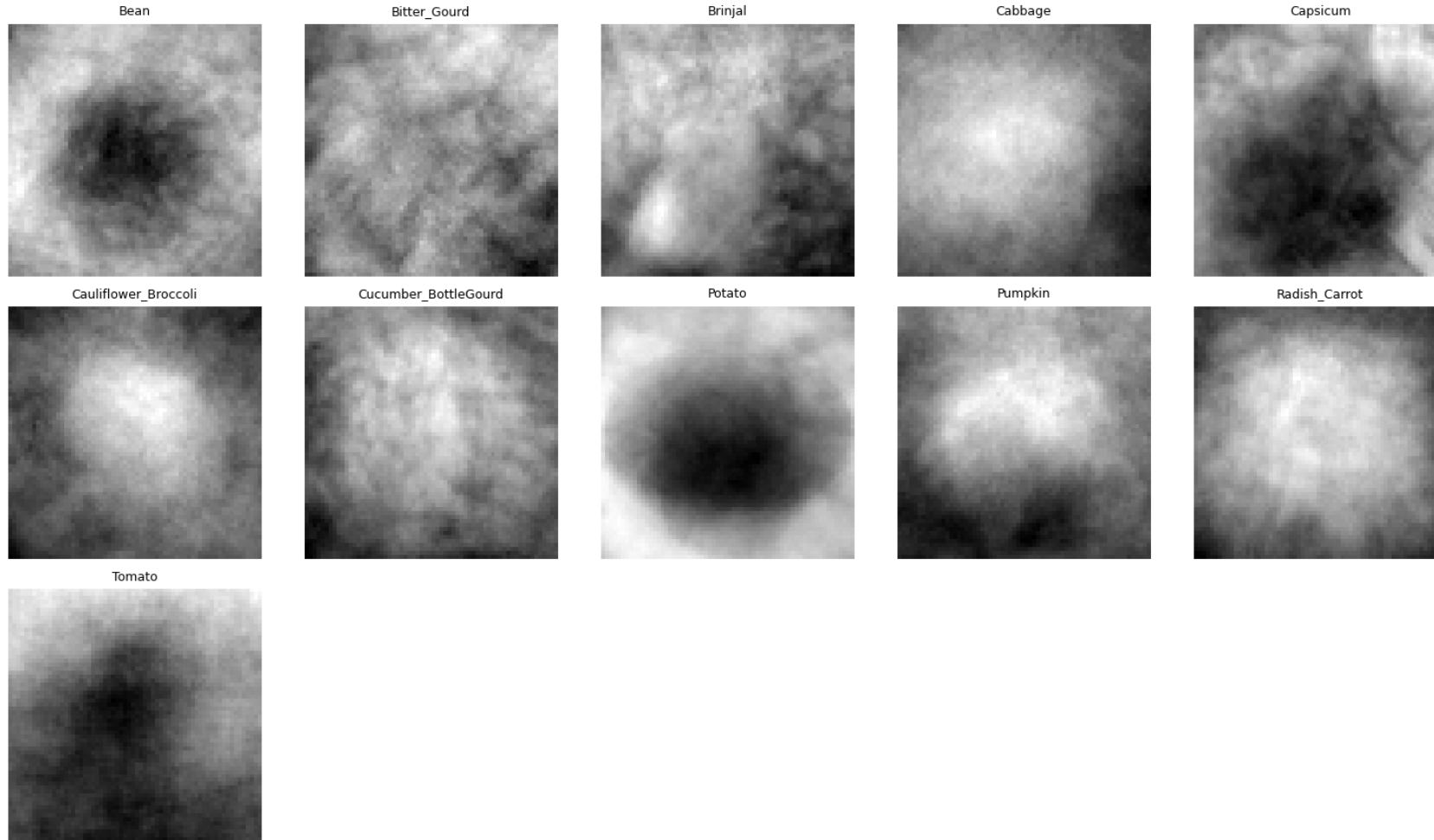
Exploratory Data Analysis



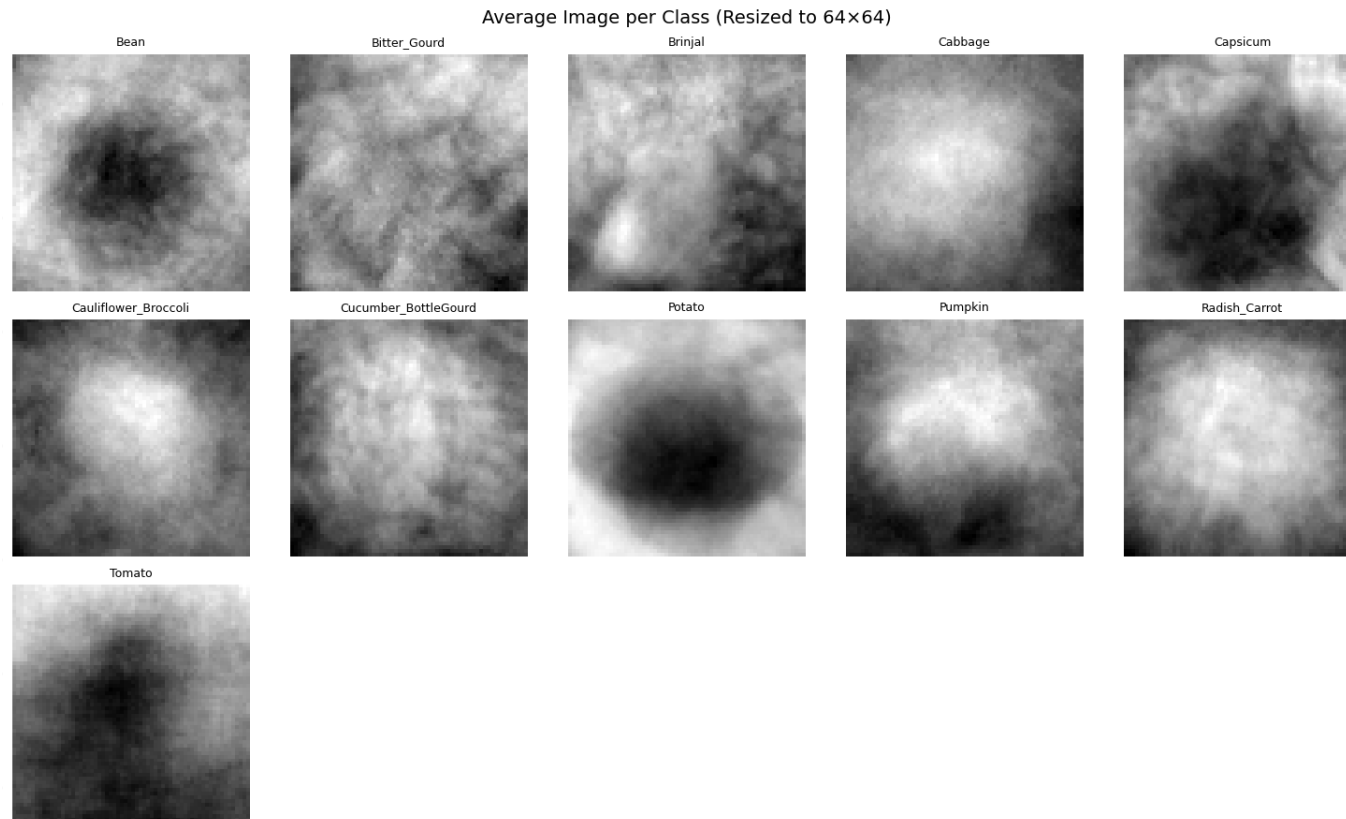
- **Left-skewed** (less dark areas, well lit)
- Most intensities lie around **150-180** (bright)
- **Less** intensities near the **high extreme** (few overexposed images)
- **No imbalance** of intensities
- **Slight brightness augmentation** can help

Exploratory Data Analysis

Average Image per Class (Resized to 64×64)



Exploratory Data Analysis



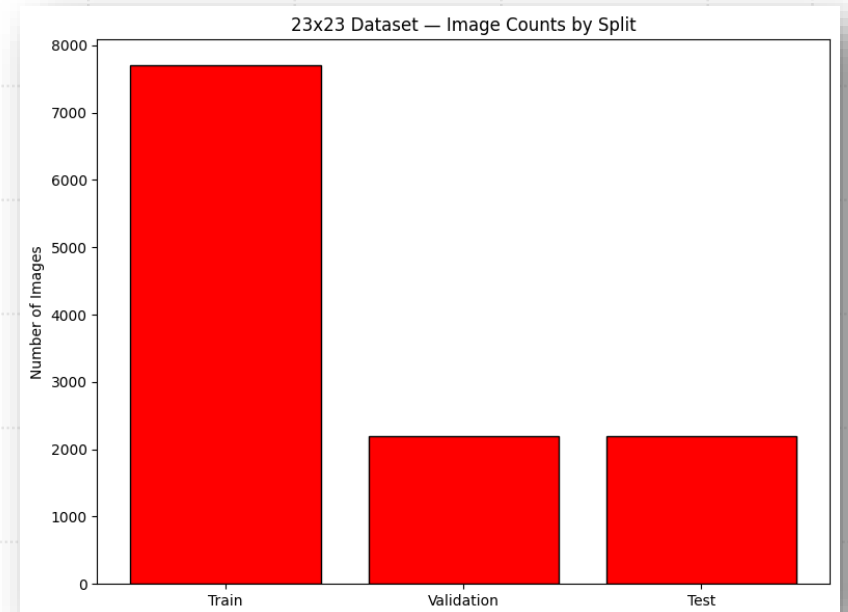
- Some images are **blurred** – vary a lot in position, orientation and scale (**brinjal**)
- Some are mostly **centered** (**potato**)
- Classes like **Radish_carrot** and **cucumber_bottlegourd** have **similar visuals** (hard to learn)
- **Dark center** (**Potato**, **Tomato**)
- This HELPS to visualize classes that the models might **struggle** to classify due to **visual similarities**

Preparing dataset for train-test-val

- Imported with `image_dataset_from_directory`
- Batch size = **128** (FIXED, not a variable in this project)
- **128** batch size – **Good balance** for our dataset size
 - **Smoother gradients** (better convergence)
 - **Balanced for speed and generalization** for our dataset size (works well with strong augmentation)
- Resized to **23x23** & **101x101**
- Stores them in the DATASET dictionary (Can add more input sizes, works dynamically with the models)

```
DATASETS = {  
    23: {  
        "train": train_ds_23,  
        "val": val_ds_23,  
        "test": test_ds_23  
    },  
    101: {  
        "train": train_ds_101,  
        "val": val_ds_101,  
        "test": test_ds_101  
    }  
}
```

```
23x23 Training – Batches: 61 | Images: 7699  
23x23 Validation – Batches: 18 | Images: 2200  
23x23 Test – Batches: 18 | Images: 2200
```



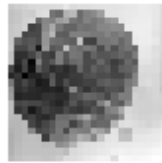
23 x 23 Grayscale Preview

23x23 Training Sample Preview

Cauliflower_Broccoli



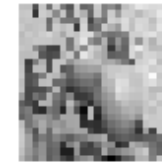
Cabbage



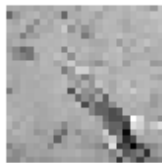
Cauliflower_Broccoli



Tomato



Bean



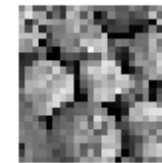
Potato



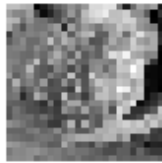
Radish_Carrot



Pumpkin



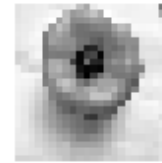
Pumpkin



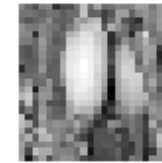
Pumpkin



Capsicum



Tomato



101 x 101 Grayscale Preview

101x101 Training Sample Preview

Cauliflower_Broccoli



Cabbage



Cauliflower_Broccoli



Tomato



Bean



Potato



Radish_Carrot



Pumpkin



Pumpkin



Pumpkin



Capsicum



Tomato



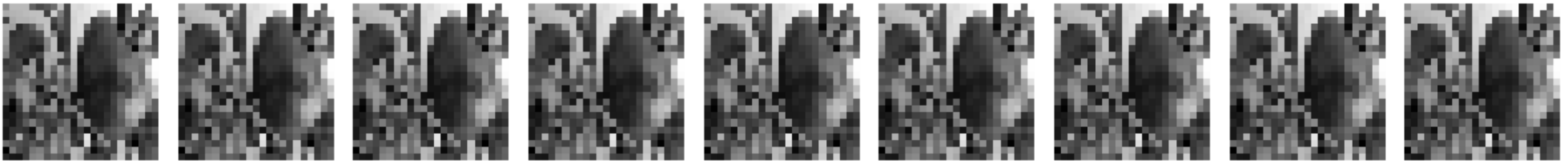
Augmentation Set-up

- Augmentations are made using `ImageDataGenerator()` – a keras utility that applies augmentation (resize/rescale/rotate/etc)
- As per requirements, usefulness of augmented data must be assessed
 - Made 2 augmentation configurations
 - **Baseline** – no augmentation applied
 - **Aggressive** – aggressive (but safe) augmentation applied

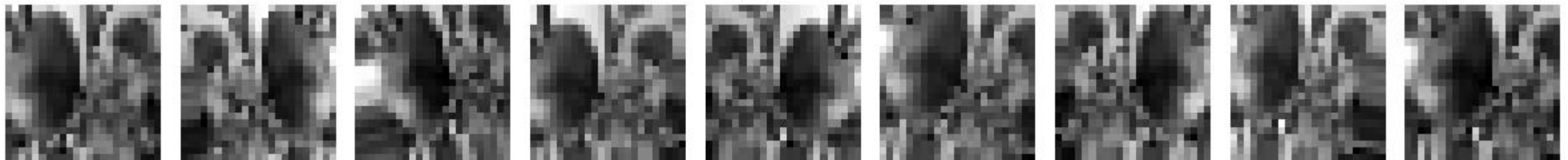
```
def get_image_generators():  
    baseline_gen = ImageDataGenerator() # no rescale  
  
    aggressive_aug_gen = ImageDataGenerator(  
        rotation_range=15,  
        width_shift_range=0.1,  
        height_shift_range=0.1,  
        zoom_range=0.1,  
        horizontal_flip=True,  
        fill_mode='nearest'  
    )  
  
    return baseline_gen, aggressive_aug_gen
```

Augmentation Preview

23x23



101x101



Defining our model's base configurations

Model 1: Baseline CNN Model

- ☐ Simple CNN with just few layers
- ☐ No tricks
- ☐ No early stopping

Model 2: Normalized CNN Model

- ☐ Added batchnormalization after each layer
- ☐ Helps train more smoothly & generalize better
- ☐ Baseline, but steadier and stable
- ☐ Uses Maxpooling2D (gets max value)

Model 3: Enhanced (Deeper) CNN Model

- ☐ Normalized model + dropout (combat overfitting)
- ☐ More complex
- ☐ More filters (capture richer features)
- ☐ Uses GlobalAvgPooling for higher res (gets average value instead of max)

Training models with manual-grid search

Testing for:

- Best **Architecture** (Baseline, Normalize, Enhanced)
- Best **optimizer** (Adam, Nadam, SGD, rmsprop)
- Best **augmentation method** (baseline, aggressive)

```
EXPERIMENT_GRID = {  
    "architectures": ["baseline", "normalize", "enhanced"],  
    "augmentations": ["baseline", "aggressive"],  
    "losses": ["ce"],  
    "optimizers": ["adam", "sgd", "rmsprop", "nadam"]  
}
```

200 Epochs with Early stopping will be run due to our complex model structures.

Manual Grid-Search vs Keras Tuner

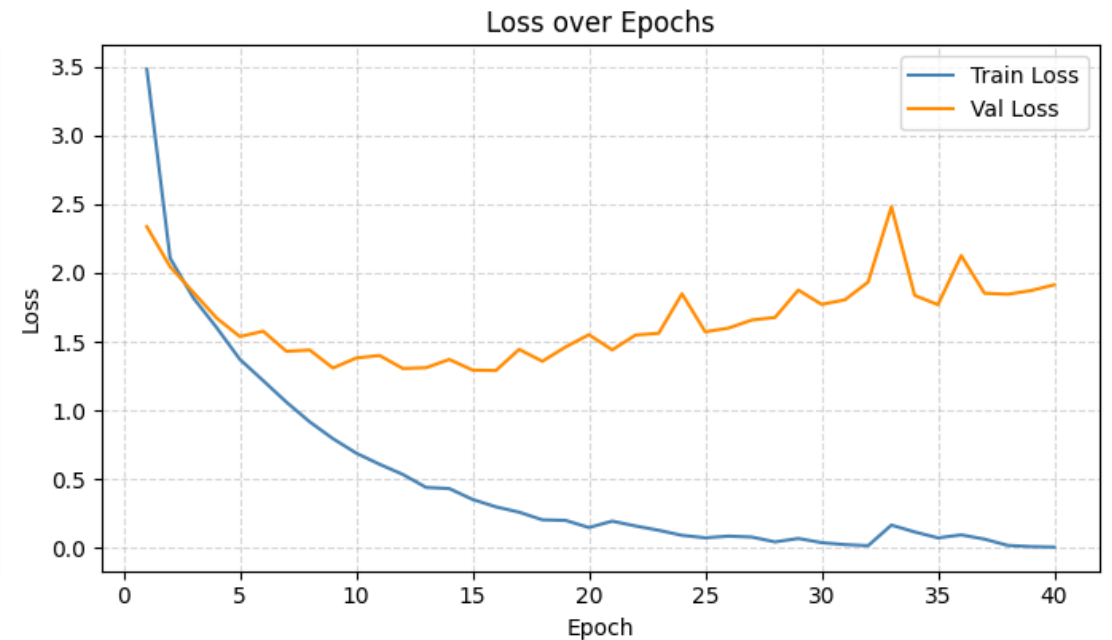
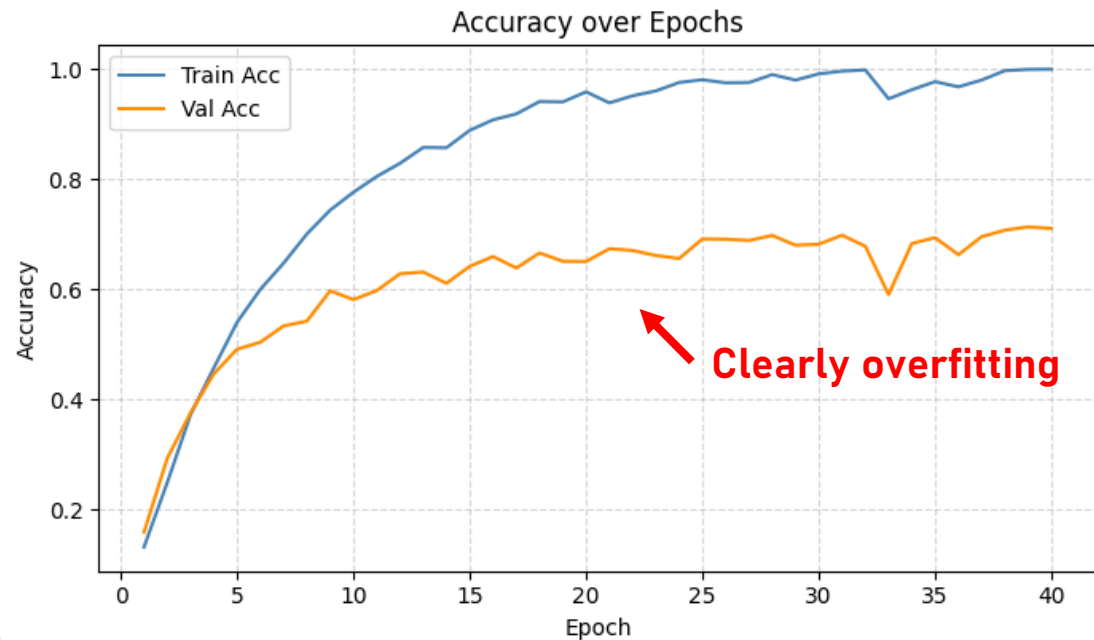
Manual grid search was chosen for its clarity, control, and reproducibility, allowing structured testing of key model configurations under a consistent evaluation pipeline.

- **Manual grid search** – full control of what configuration and what model to test – no hidden search logic
- All combinations and results are **logged (and exported to a csv file)**
- **Keras-Tuner** does all the tuning for you **without showing much reasoning** on why other parameters are not compatible **VS Manual Grid Search** able to show every configuration's **performance**.
- In short, manual grid search offers a **more targeted parameter search** as well as **more insights into what configurations work for this dataset**, while **sacrificing little performance**.

Model Training Results – Baseline

Model	Input Size	Augmentation	Loss	Optimizer	Train accuracy	Validation Accuracy	Test accuracy
Baseline	23x23	Baseline	ce	Adam	99%	71%	72%

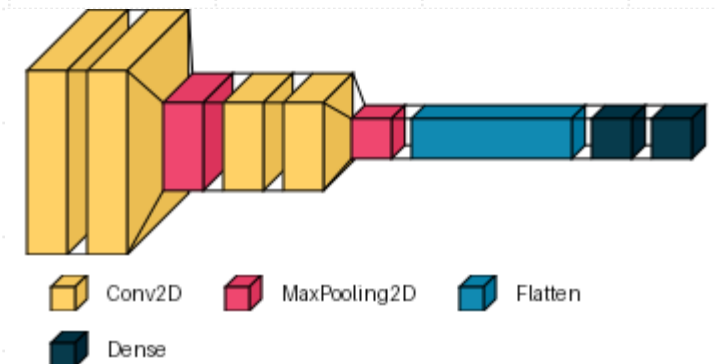
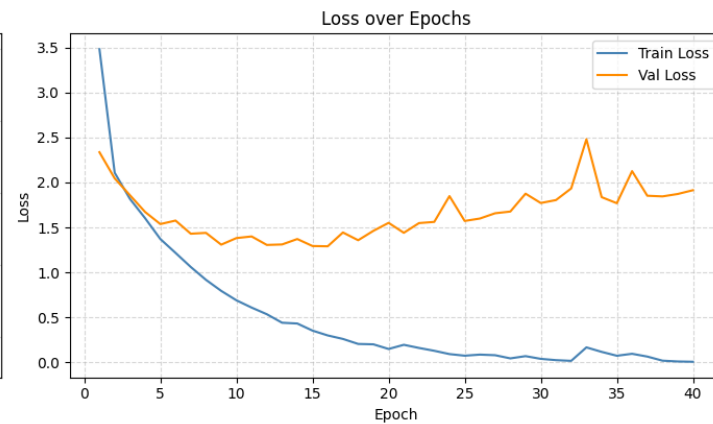
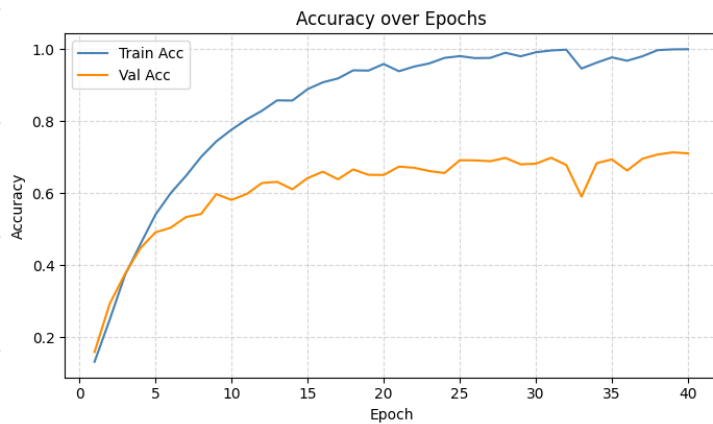
baseline_baseline_ce_adam_None_23



Model Training Results – Baseline

Model	Input Size	Augmentation	Loss	Optimizer	Train accuracy	Validation Accuracy	Test accuracy
Baseline	23x23	Baseline	ce	Adam	99%	71%	72%

baseline_baseline_ce_adam_None_23

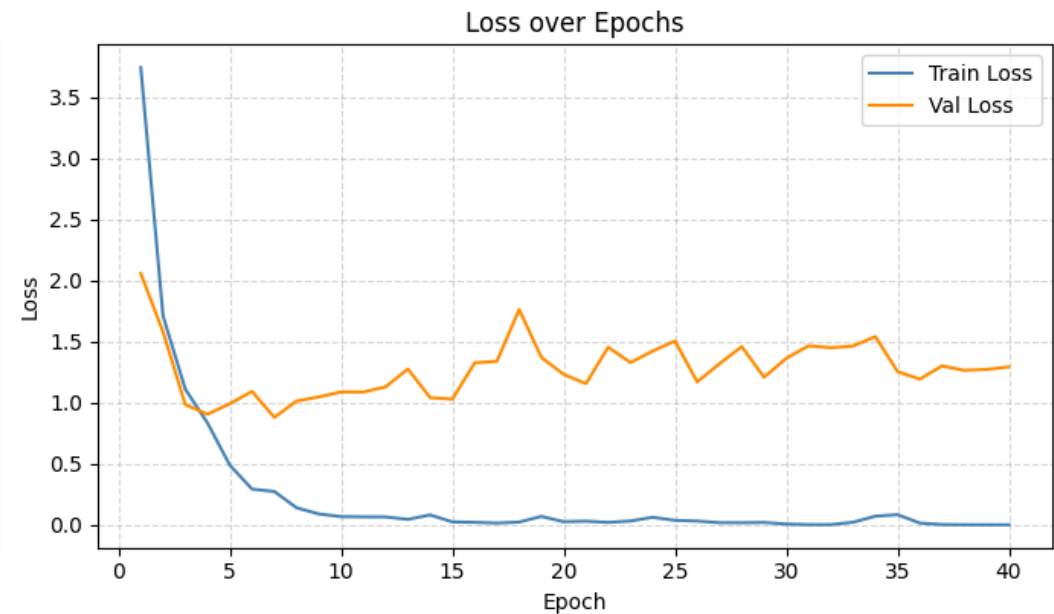
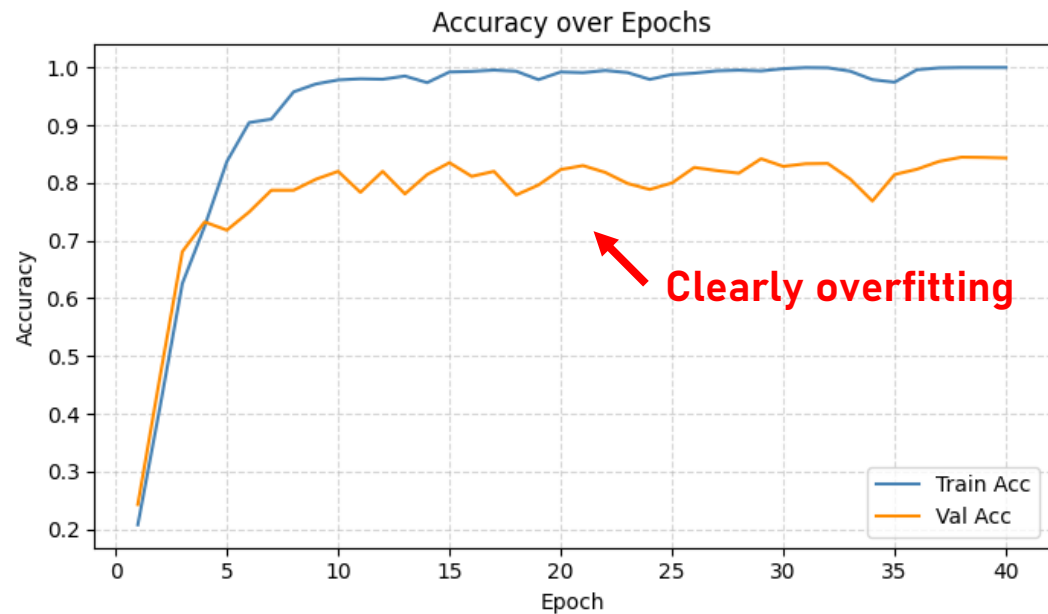


VisualKeras-style architecture
for baseline 23x23

Model Training Results – Baseline

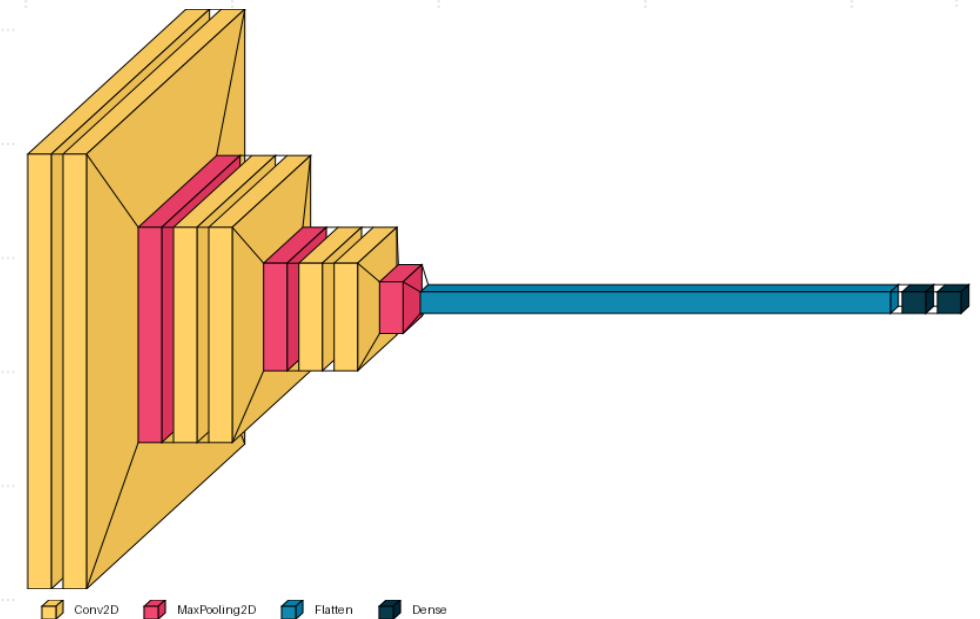
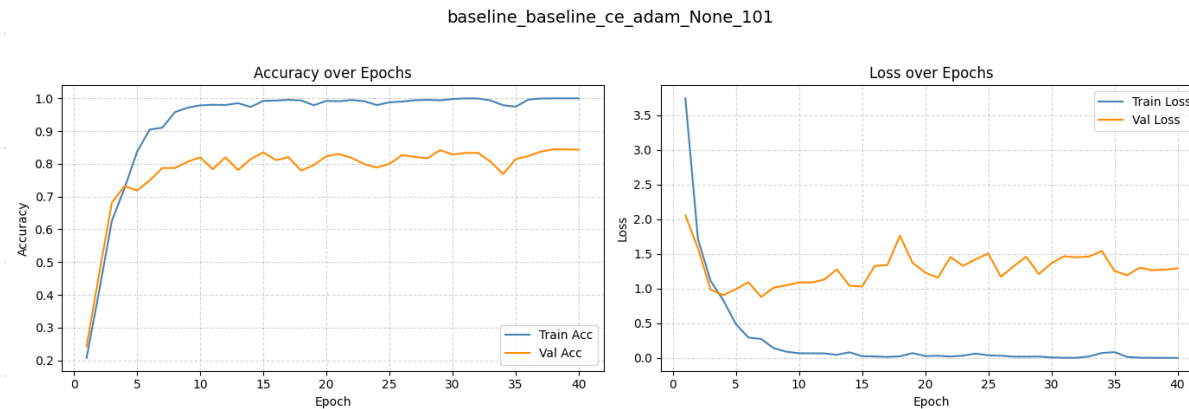
Model	Input Size	Augmentation	Loss	Optimizer	Train accuracy	Validation Accuracy	Test accuracy
Baseline	101x101	Baseline	ce	Adam	100%	84.4%	84.7%

baseline_baseline_ce_adam_None_101



Model Training Results – Baseline

Model	Input Size	Augmentation	Loss	Optimizer	Train accuracy	Validation Accuracy	Test accuracy
Baseline	101x101	Baseline	ce	Adam	100%	84.4%	84.7%

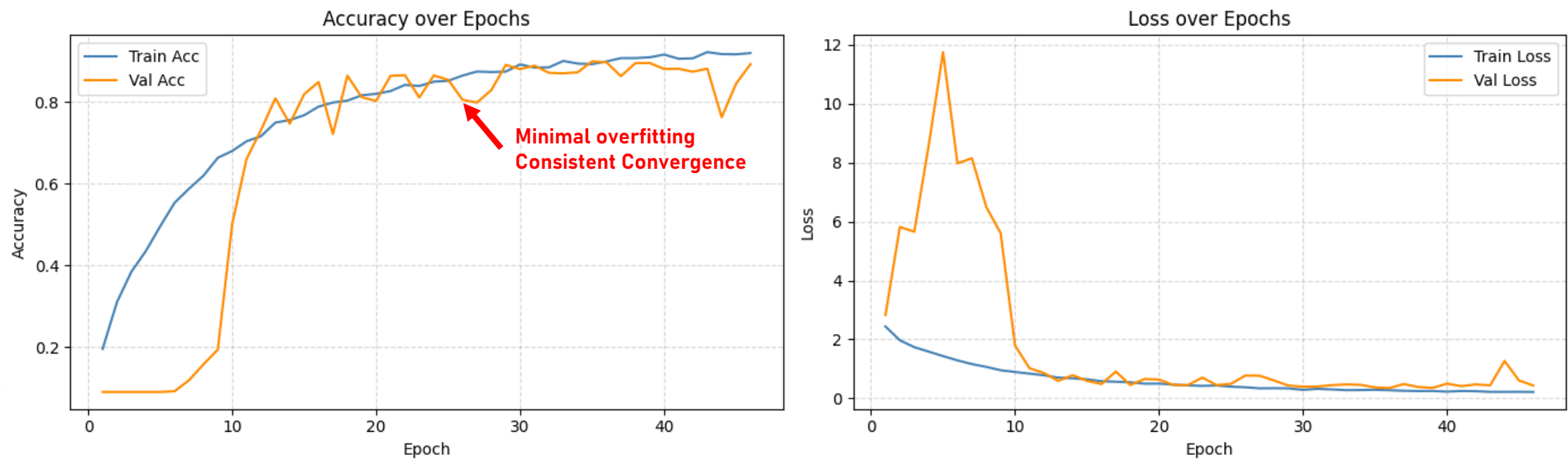


VisualKeras-style architecture
for baseline 101x101

Model Training Results – Normalized (top 3)

Model	Input Size	Augmentation	Loss	Optimizer	Train accuracy	Validation Accuracy	Test accuracy
Normalized	23x23	Aggressive	Ce	Adam	92.3%	90.0%	90.8%
Normalized	23x23	Aggressive	Ce	Sgd	88.7%	89.8%	89.8%
Normalized	23x23	Baseline	Ce	Nadam	91.5%	88.6%	88.3%

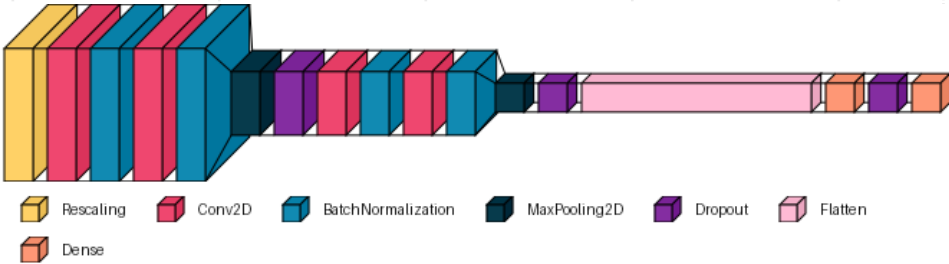
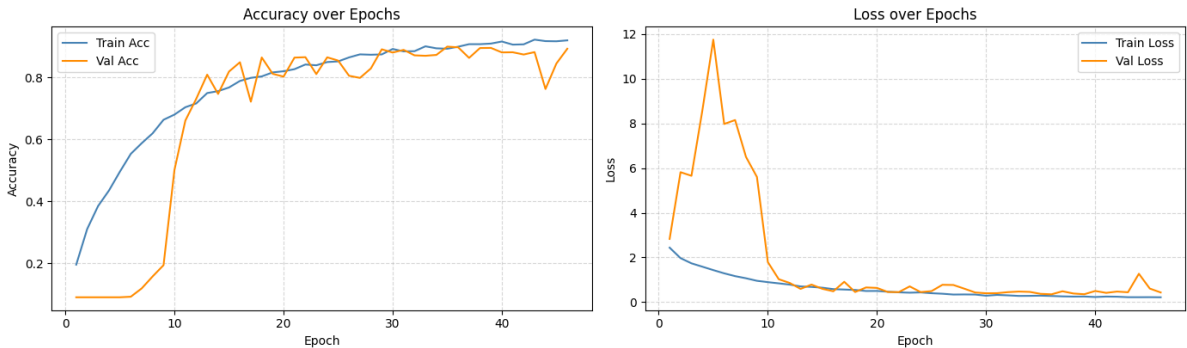
normalize_aggressive_ce_adam_None_23



Model Training Results – Normalized (top 3)

Model	Input Size	Augmentation	Loss	Optimizer	Train accuracy	Validation Accuracy	Test accuracy
Normalized	23x23	Aggressive	Ce	Adam	92.3%	90.0%	90.8%
Normalized	23x23	Aggressive	Ce	Sgd	88.7%	89.8%	89.8%
Normalized	23x23	Baseline	Ce	Nadam	91.5%	88.6%	88.3%

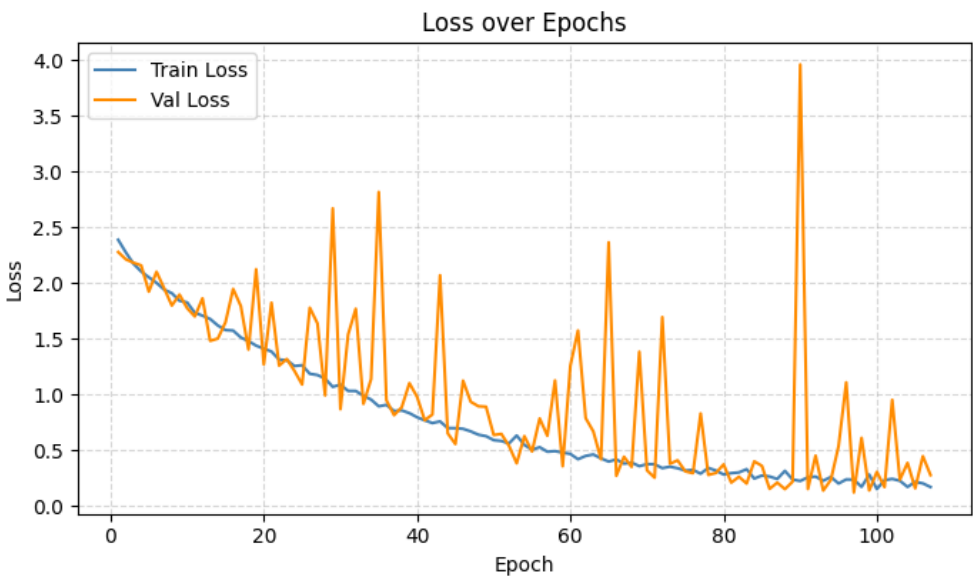
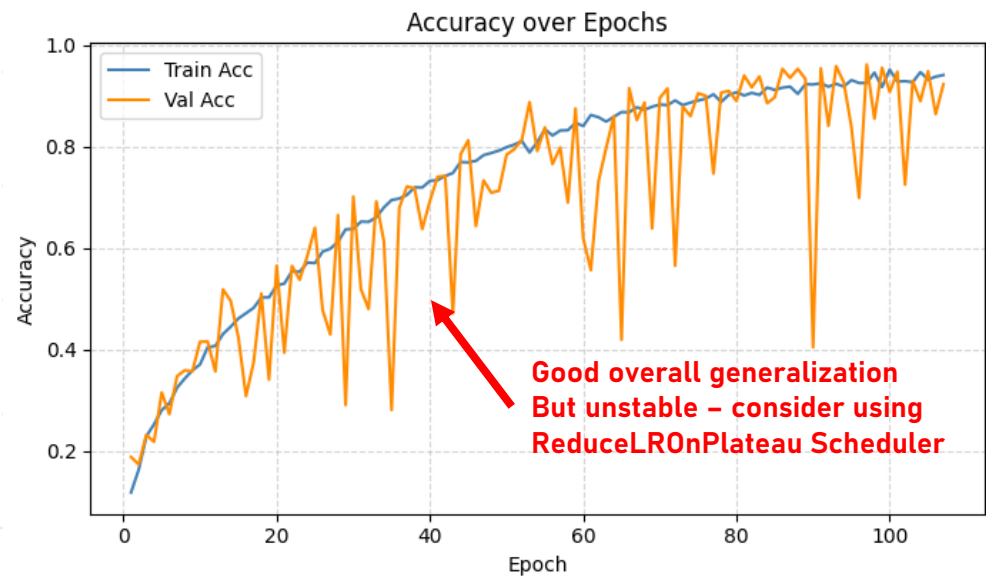
normalize_aggressive_ce_adam_None_23



Model Training Results – Normalized (top 3)

Model	Input Size	Augmentation	Loss	Optimizer	Train accuracy	Validation Accuracy	Test accuracy
Normalized	101x101	baseline	Ce	Rmsprop	95.2%	96.2%	96.3%
Normalized	101x101	Aggressive	Ce	Rmsprop	93.4%	95.0%	95.9%
Normalized	101x101	Aggressive	Ce	Nadam	94.2%	94.9%	95.5%

normalize_baseline_ce_rmsprop_None_101



A green triangle is plotted on a coordinate plane. The vertices are located at the coordinates $(-1, 0)$, $(-1, 2)$, and $(1, 1)$. The triangle is shaded in a solid green color.

The figure consists of three subplots related to the training of a neural network model.

Accuracy over Epochs: This line plot shows the training accuracy (Train Acc, blue line) and validation accuracy (Val Acc, orange line) over 100 epochs. The training accuracy starts at approximately 0.15 and increases steadily to about 0.95. The validation accuracy starts at approximately 0.25 and increases to about 0.9, showing some fluctuations.

Loss over Epochs: This line plot shows the training loss (Train Loss, blue line) and validation loss (Val Loss, orange line) over 100 epochs. The training loss starts at approximately 2.4 and decreases steadily to about 0.2. The validation loss starts at approximately 2.4 and decreases to about 0.2, with a significant spike around epoch 90.

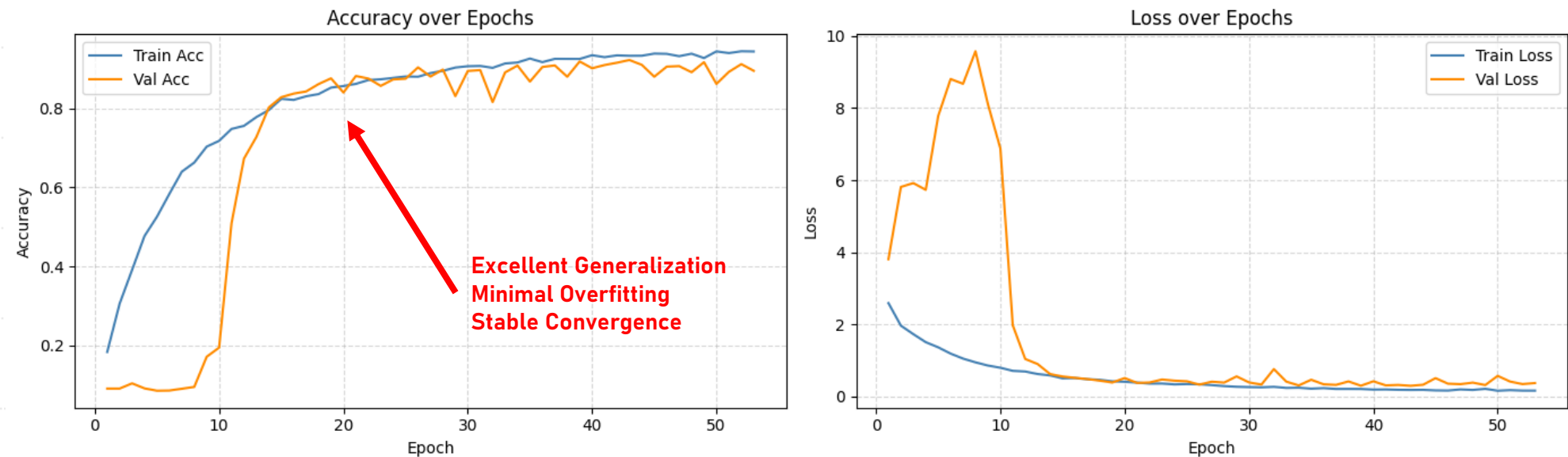
Model Architecture: A 3D visualization of the model architecture is shown on the right. The layers are represented by colored blocks: Rescaling (yellow), Conv2D (red), MaxPooling2D (blue), Dropout (dark blue), GlobalAveragePooling2D (purple), and Dense (pink). The architecture starts with a Rescaling layer, followed by three Conv2D layers, three MaxPooling2D layers, two Dropout layers, a GlobalAveragePooling2D layer, and finally a Dense layer.



Model Training Results – Enhanced (top 3)

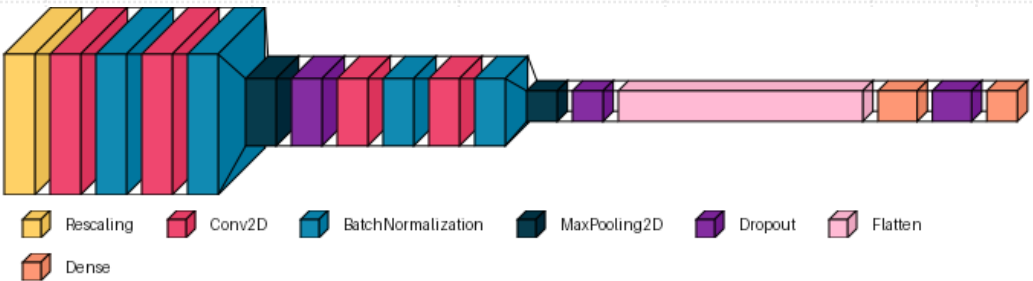
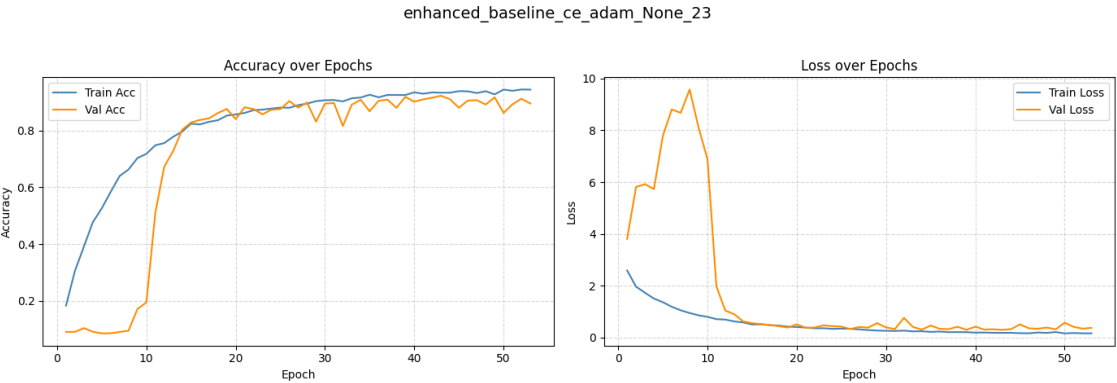
Model	Input Size	Augmentation	Loss	Optimizer	Train accuracy	Validation Accuracy	Test accuracy
Enhanced	23x23	Baseline	Ce	Adam	94.4%	92.2%	92.0%
Enhanced	23x23	Aggressive	Ce	Nadam	93.5%	91.5%	90.9%
Enhanced	23x23	Baseline	Ce	Nadam	93.6%	91.4%	91.2%

enhanced_baseline_ce_adam_None_23



Model Training Results – Enhanced (top 3)

Model	Input Size	Augmentation	Loss	Optimizer	Train accuracy	Validation Accuracy	Test accuracy
Enhanced	23x23	Baseline	Ce	Adam	94.4%	92.2%	92.0%
Enhanced	23x23	Aggressive	Ce	Nadam	93.5%	91.5%	90.9%
Enhanced	23x23	Baseline	Ce	Nadam	93.6%	91.4%	91.2%

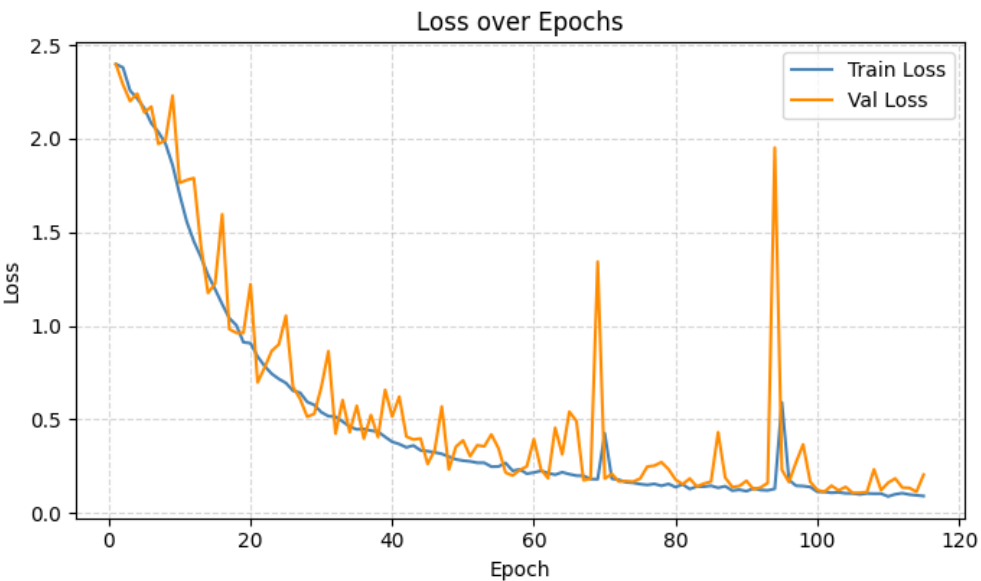
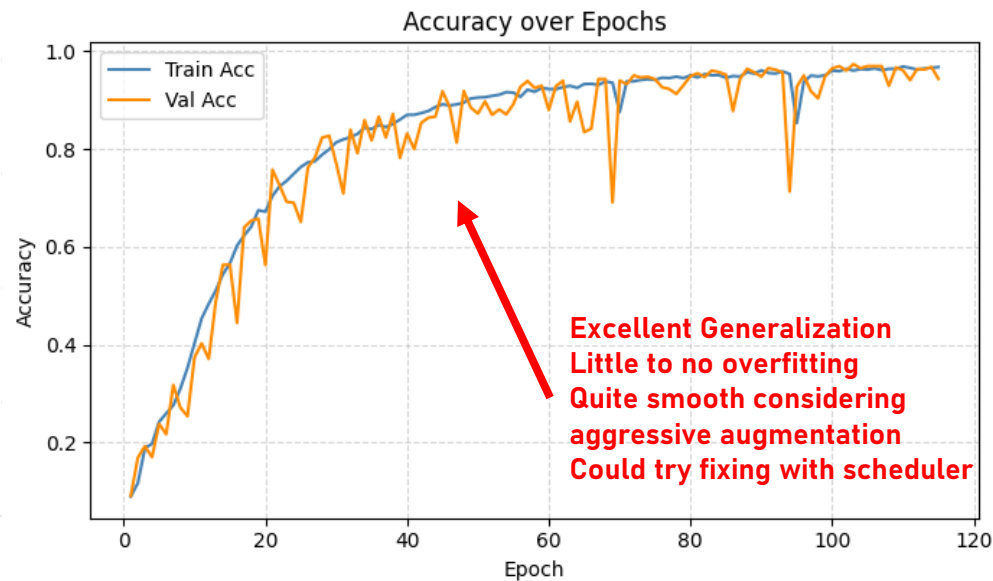


VisualKeras-style architecture
For enhanced 23x23

Model Training Results – Enhanced (top 3)

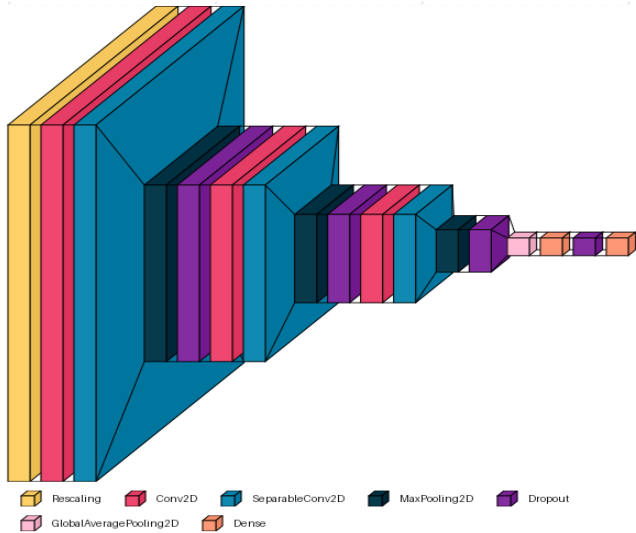
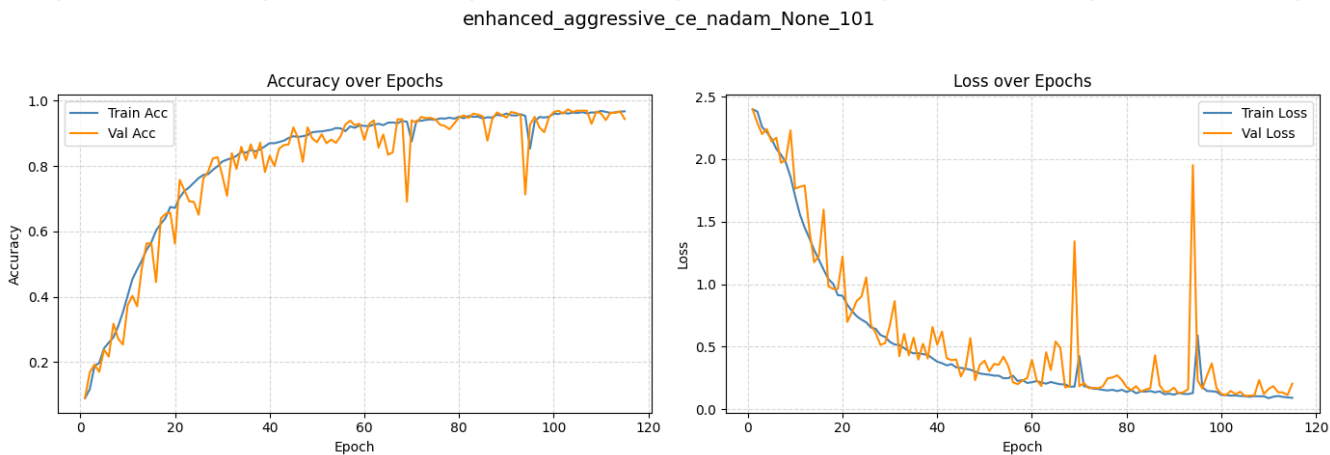
Model	Input Size	Augmentation	Loss	Optimizer	Train accuracy	Validation Accuracy	Test accuracy
Enhanced	101x101	Aggressive	Ce	Nadam	96.9%	97.4%	96.7%
Enhanced	101x101	Baseline	Ce	Nadam	96.4%	96.8%	96.8%
Enhanced	101x101	Aggressive	Ce	Adam	95.4%	96.5%	96.6%

enhanced_aggressive_ce_nadam_None_101



Model Training Results – Enhanced (top 3)

Model	Input Size	Augmentation	Loss	Optimizer	Train accuracy	Validation Accuracy	Test accuracy
Enhanced	101x101	Aggressive	Ce	Nadam	96.9%	97.4%	96.7%
Enhanced	101x101	Baseline	Ce	Nadam	96.4%	96.8%	96.8%
Enhanced	101x101	Aggressive	Ce	Adam	95.4%	96.5%	96.6%



VisualKeras-style architecture
For enhanced 101x101

Overall Top 3 for each resolution

Based on mean accuracy across train/val/test

Model	Input Size	Augmentation	Loss	Optimizer	Train accuracy	Validation Accuracy	Test accuracy
Enhanced	23x23	Baseline	Ce	Adam	94.4%	92.2%	92.0%
Enhanced	23x23	Baseline	Ce	Nadam	93.6%	91.4%	91.2%
Enhanced	23x23	Aggressive	Ce	Adam	93.5%	91.5%	90.9%

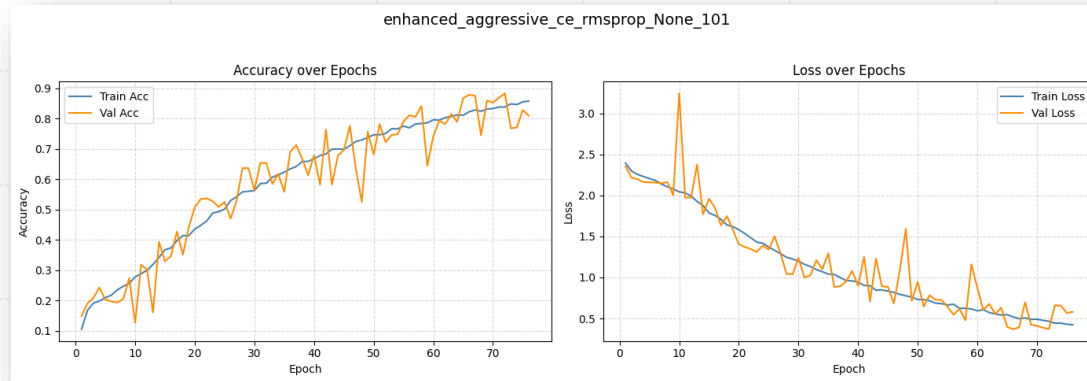
Model	Input Size	Augmentation	Loss	Optimizer	Train accuracy	Validation Accuracy	Test accuracy
Enhanced	101x101	Aggressive	Ce	Nadam	96.9%	97.4%	96.7%
Enhanced	101x101	Baseline	Ce	Nadam	96.4%	96.8%	96.8%
Enhanced	101x101	Aggressive	Ce	Adam	95.4%	96.5%	96.6%

Deeper & Stronger regularization
Enhanced Models – Superior

Best 23x23 Model Accuracy	Best 101x101 Model Accuracy
92%	~96%

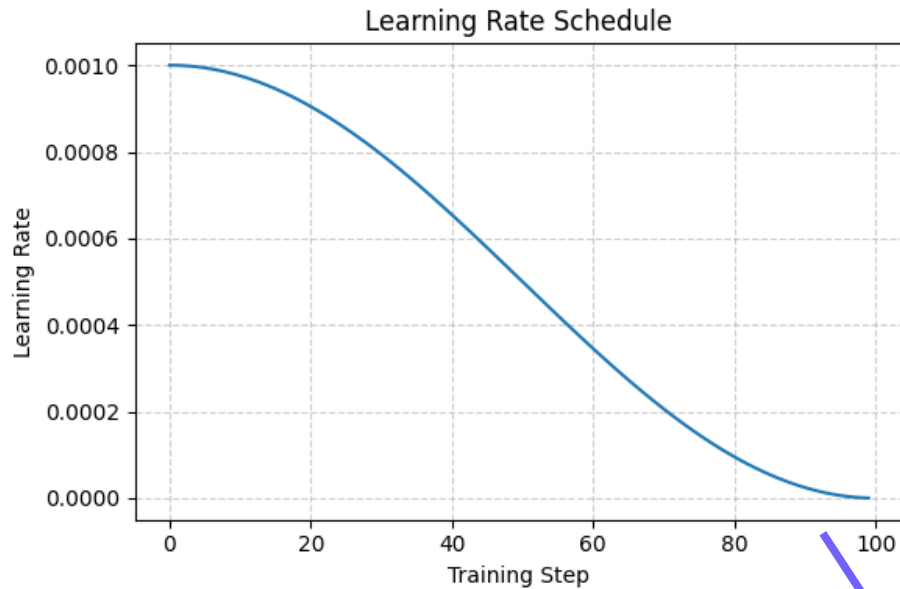
Further Model Improvement Experimentation - Schedulers

- Seen from earlier – Unstable learning curves.

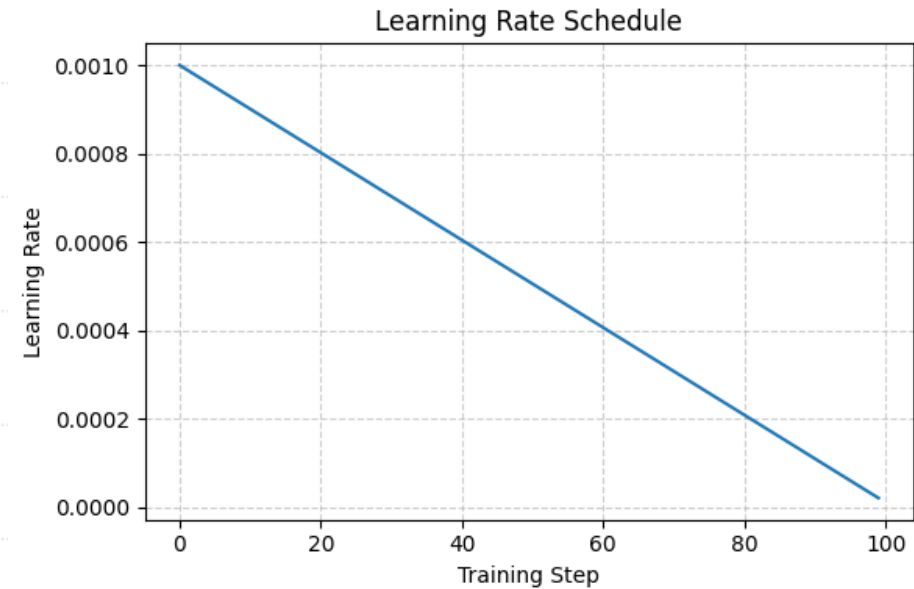


- Applied Schedulers to the top 5 models of 23x23 and 101x101
 - **Cosine Decay** (Learning Rate decreases following a cosine curve)
 - **Polynomial Decay** (Learning rate decreases by pre-set steps)
 - **ReduceLROnPlateau** (Most stable for small datasets)
- Trial'd with Plateau only for 101x101 due to bad performance of other schedulers on 23x23 (saves computing resources)

Visualization for Cosine Decay & Polynomial Decay

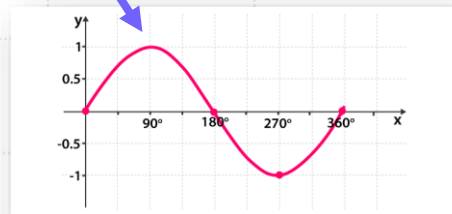


Cosine Decay



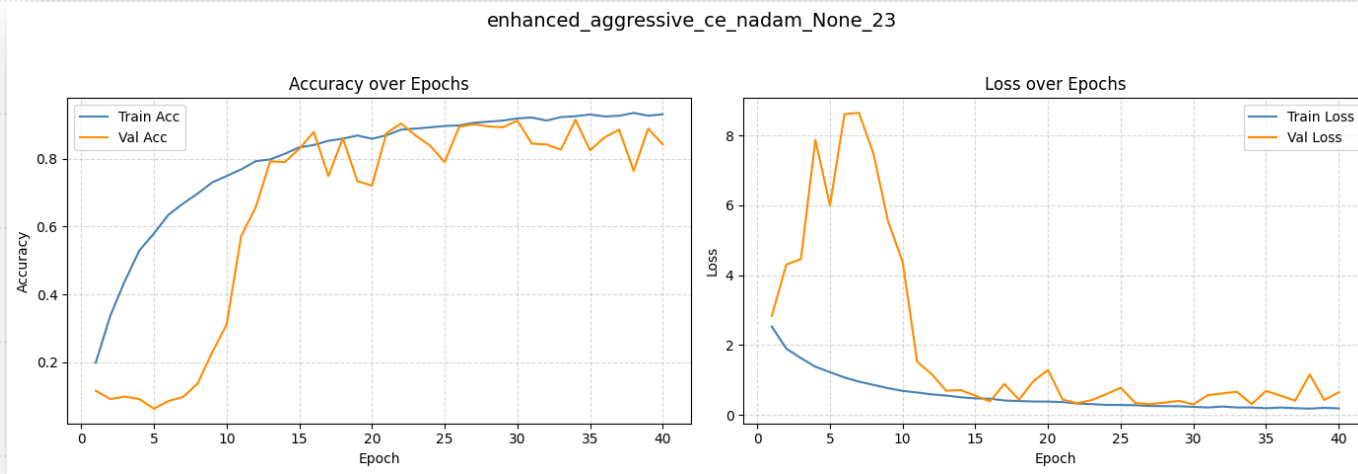
Polynomial Decay

Just like a cosine graph!



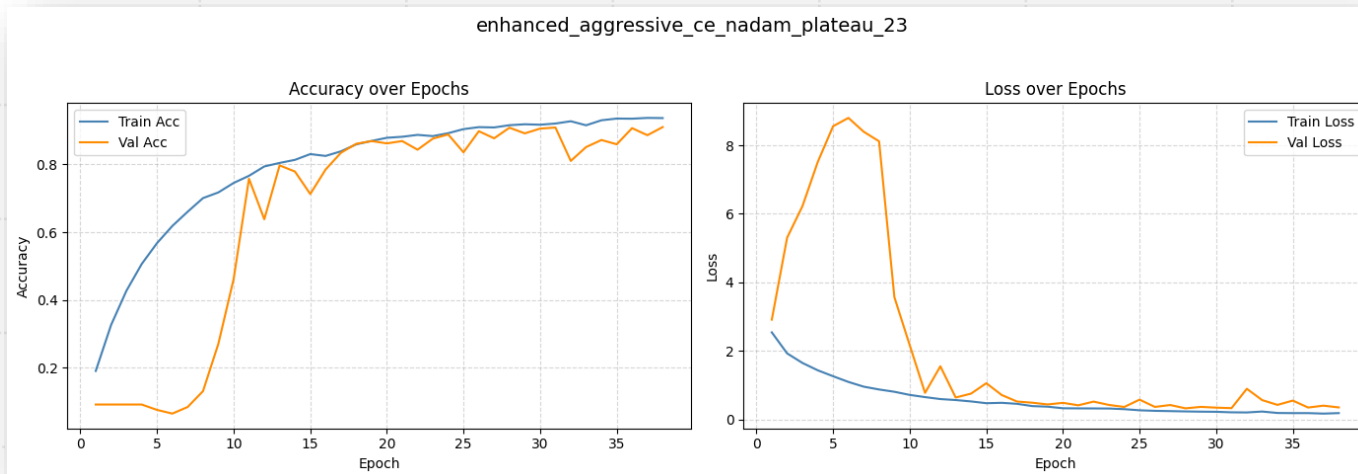
Scheduler Trial for 23x23 – Plateau

BEFORE



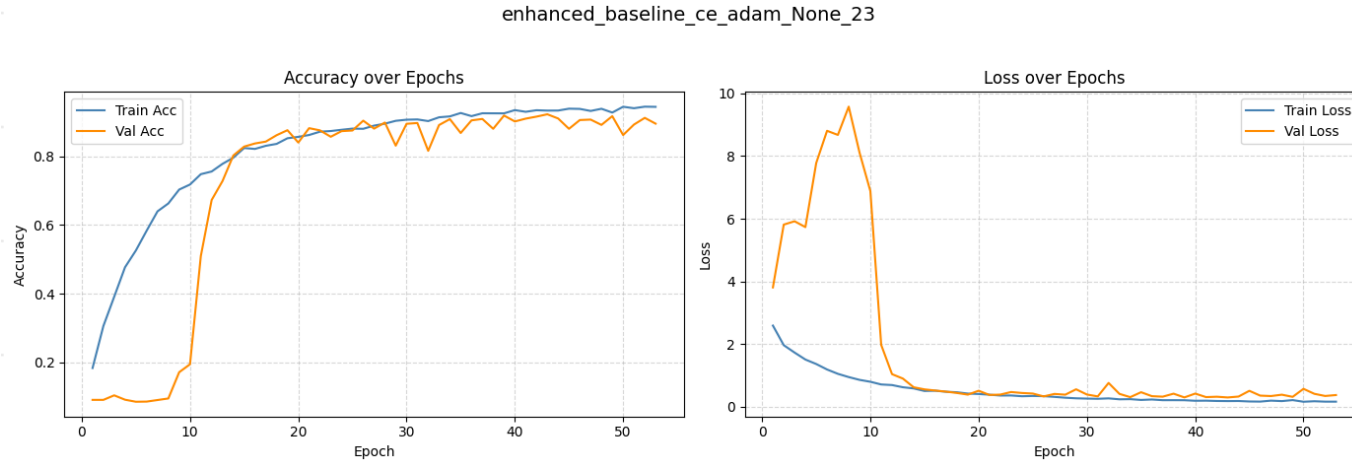
✓ Stabilized
convergence

AFTER



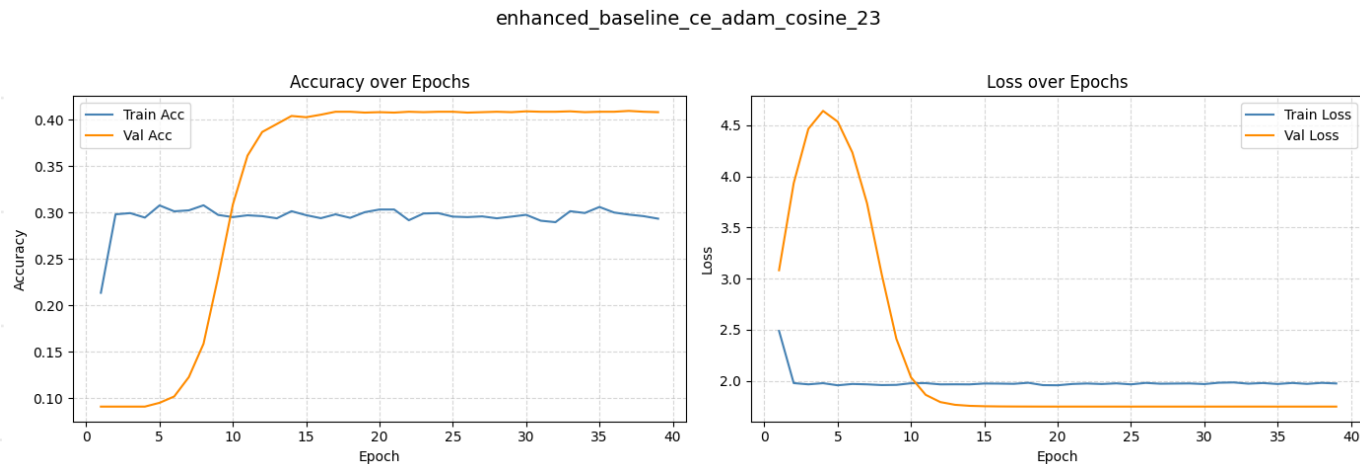
Scheduler Trial for 23x23 - Cosine

BEFORE



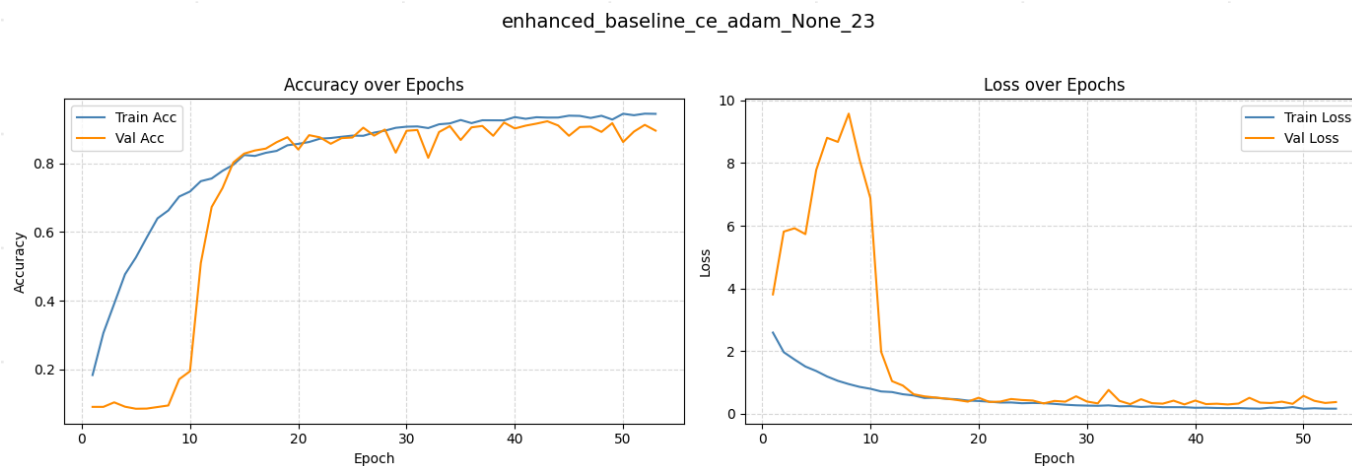
× Completely broke the model

AFTER



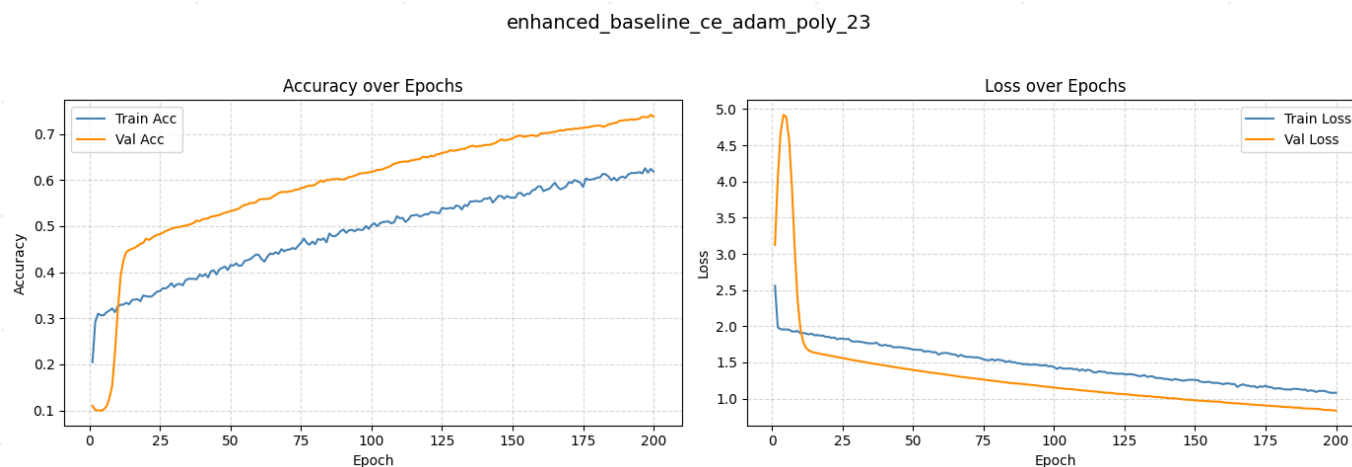
Scheduler Trial for 23x23 - Poly

BEFORE



× Sacrificed performance and underfitted for stabilized curve.

AFTER



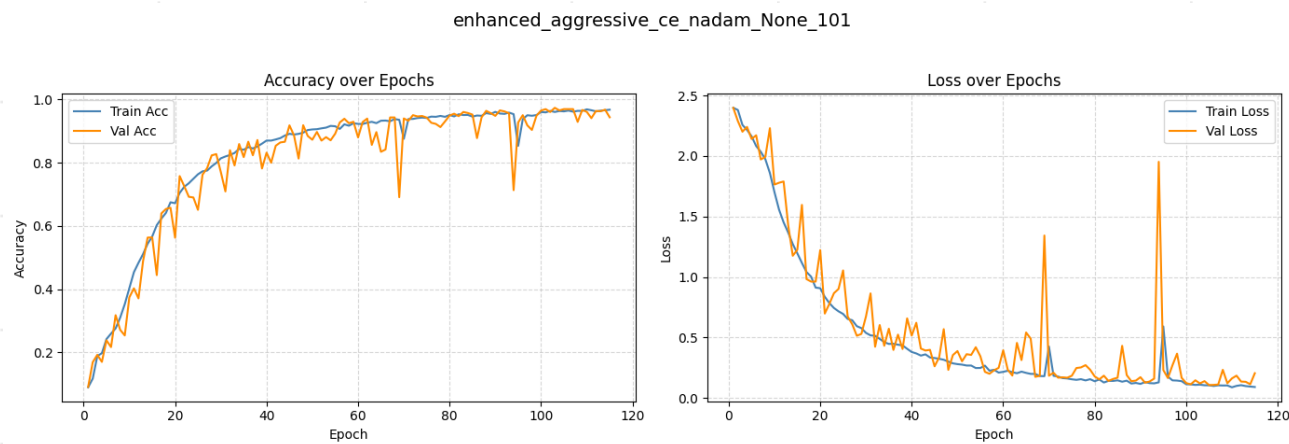
Overall scheduler performances for 23x23

	Model	InputSize	Augmentation	Loss	Optimizer	Accuracy	Val_Accuracy	LossFinal	Val_LossFinal	RuntimeSec	Test_Accuracy
0	enhanced_baseline_ce_adam_plateau_23	23	baseline	ce	adam	0.9467	0.9177	0.1526	0.4073	71.90	0.9223
1	enhanced_baseline_ce_nadam_plateau_23	23	baseline	ce	nadam	0.9367	0.9100	0.1787	0.4208	67.91	0.9191
2	enhanced_aggressive_ce_nadam_plateau_23	23	aggressive	ce	nadam	0.9375	0.9105	0.1846	0.3505	60.15	0.9141
3	enhanced_aggressive_ce_rmsprop_plateau_23	23	aggressive	ce	rmsprop	0.9105	0.8836	0.2650	0.4701	42.66	0.8732
4	normalize_aggressive_ce_adam_plateau_23	23	aggressive	ce	adam	0.8614	0.8509	0.3833	0.6262	40.96	0.8514
5	enhanced_aggressive_ce_rmsprop_poly_23	23	aggressive	ce	rmsprop	0.6418	0.7464	1.0470	0.7875	321.77	0.7418
6	enhanced_baseline_ce_adam_poly_23	23	baseline	ce	adam	0.6255	0.7414	1.0821	0.8348	323.54	0.7259
7	normalize_aggressive_ce_adam_poly_23	23	aggressive	ce	adam	0.5744	0.7027	1.2137	0.9288	316.81	0.7000
8	enhanced_aggressive_ce_rmsprop_cosine_23	23	aggressive	ce	rmsprop	0.3559	0.4691	1.8825	1.5914	58.35	0.4955
9	enhanced_baseline_ce_adam_cosine_23	23	baseline	ce	adam	0.3078	0.4095	1.9740	1.7479	68.20	0.4050
10	normalize_aggressive_ce_adam_cosine_23	23	aggressive	ce	adam	0.2521	0.3664	2.0871	1.9026	63.51	0.3518

Our accuracy scores further prove that only ReduceLRonPlateau has potential for improving the models, while the rest breaks the model.

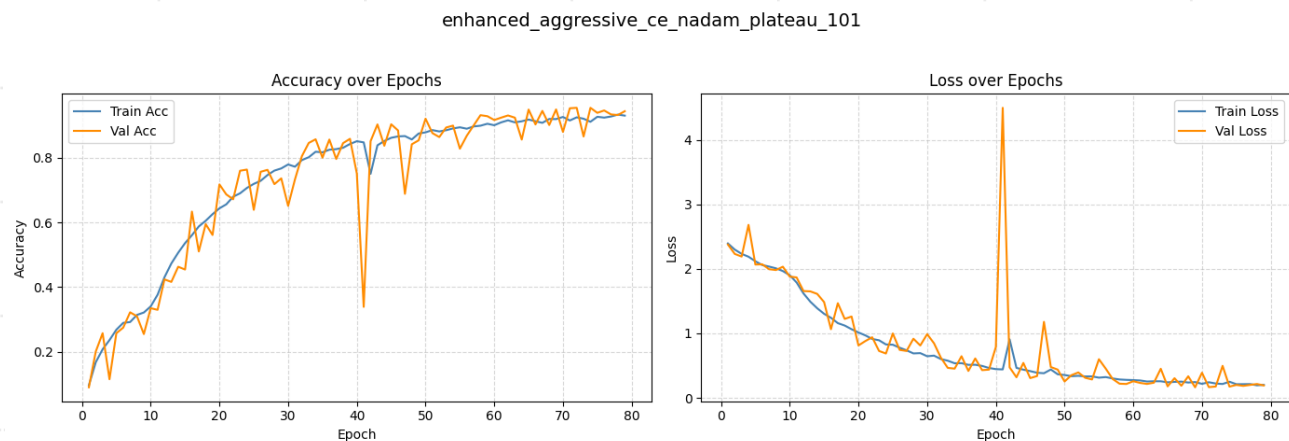
Scheduler Trial for 101x101 - Plateau

BEFORE



× Did not help,
instead
caused more
instability.

AFTER



Comparison for use of schedulers

Schedulers



	Model	InputSize	Scheduler	Val_Accuracy	Test_Accuracy	Macro_F1	Weighted_F1
0	enhanced_baseline_ce_adam_None_23	23	None	0.9223	0.9200	0.9201	0.9201
1	enhanced_baseline_ce_nadam_None_23	23	None	0.9136	0.9118	0.9124	0.9124
2	enhanced_aggressive_ce_nadam_None_23	23	None	0.9150	0.9095	0.9098	0.9098
→ 3	enhanced_baseline_ce_adam_plateau_23	23	Plateau	0.9177	0.9223	0.9230	0.9230
→ 4	enhanced_baseline_ce_nadam_plateau_23	23	Plateau	0.9100	0.9191	0.9187	0.9187
→ 5	enhanced_aggressive_ce_nadam_plateau_23	23	Plateau	0.9105	0.9141	0.9139	0.9139

Schedulers



	Model	InputSize	Scheduler	Val_Accuracy	Test_Accuracy	Macro_F1	Weighted_F1
0	enhanced_aggressive_ce_nadam_None_101	101	None	0.9736	0.9668	0.9668	0.9668
1	enhanced_baseline_ce_nadam_None_101	101	None	0.9682	0.9677	0.9677	0.9677
2	enhanced_aggressive_ce_adam_None_101	101	None	0.9650	0.9659	0.9659	0.9659
→ 3	normalize_aggressive_ce_nadam_plateau_101	101	Plateau	0.9541	0.9545	0.9545	0.9545
→ 4	enhanced_baseline_ce_nadam_plateau_101	101	Plateau	0.9559	0.9550	0.9550	0.9550
→ 5	enhanced_aggressive_ce_nadam_plateau_101	101	Plateau	0.9541	0.9491	0.9490	0.9490

Plateau Schedulers benefitted 23x23 models, but decreased performance for 101x101 models.

Final Comparison – Best overall models

Model	Input Size	Augmentation	Loss	Optimizer	Scheduler	Train Accuracy	Validation Accuracy	Test Accuracy	Macro-F1 Score	Weighted-F1 Score
Enhanced	23x23	Baseline	CE	Adam	Plateau	94.7%	91.8%	92.2%	92.3%	92.3%

Model	Input Size	Augmentation	Loss	Optimizer	Scheduler	Train Accuracy	Validation Accuracy	Test Accuracy	Macro-F1 Score	Weighted-F1 Score
Enhanced	101x101	Aggressive	CE	Nadam	None	96.9%	97.4%	96.7%	96.6%	96.6%

Our models perform exceptionally well, with incredible F1 Scores, signifying exceptional class prediction.

Same F1 Scores – no bias to any class, even if imbalanced.

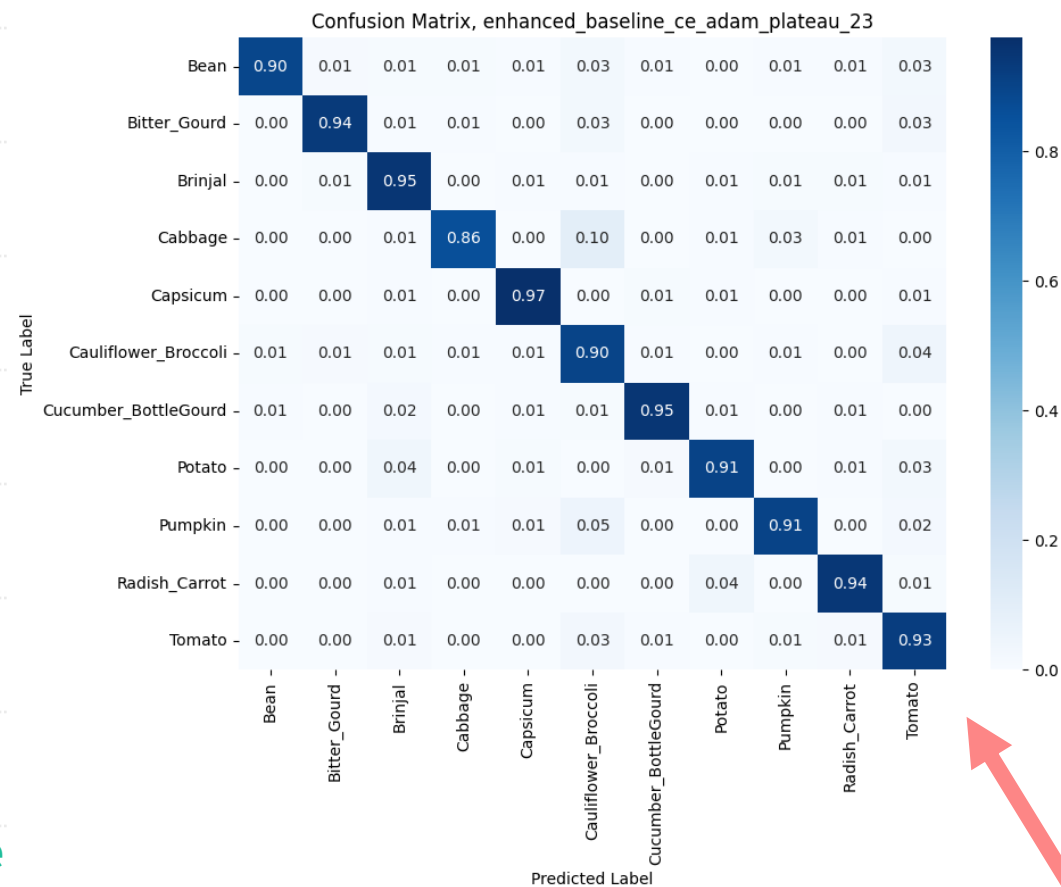
Augmentation harms performance for 23x23 – Logical, as augmenting low resolutions distorts the already minimal features

Augmentation helps for 101x101 – Logical, as augmenting higher resolutions wouldn't harm, but helps create intra-class variance

Reliability test – Classification report + Normalized Confusion matrix

Best model of 23x23

Classification Report for: enhanced_baseline_ce_adam_plateau_23				
	precision	recall	f1-score	support
Bean	0.98	0.90	0.94	200
Bitter_Gourd	0.96	0.94	0.95	200
Brinjal	0.89	0.95	0.92	200
Cabbage	0.97	0.86	0.91	200
Capsicum	0.96	0.97	0.97	200
Cauliflower_Broccoli	0.79	0.90	0.84	200
Cucumber_BottleGourd	0.96	0.95	0.95	200
Potato	0.94	0.91	0.92	200
Pumpkin	0.94	0.91	0.92	200
Radish_Carrot	0.95	0.94	0.95	200
Tomato	0.85	0.93	0.89	200
accuracy			0.92	2200
macro avg	0.93	0.92	0.92	2200
weighted avg	0.93	0.92	0.92	2200



Normalized Confusion matrix – common industry practice way to show class-based prediction, much clearer and easier to compare

- Shows as a percentage of correct predictions instead of actual size.

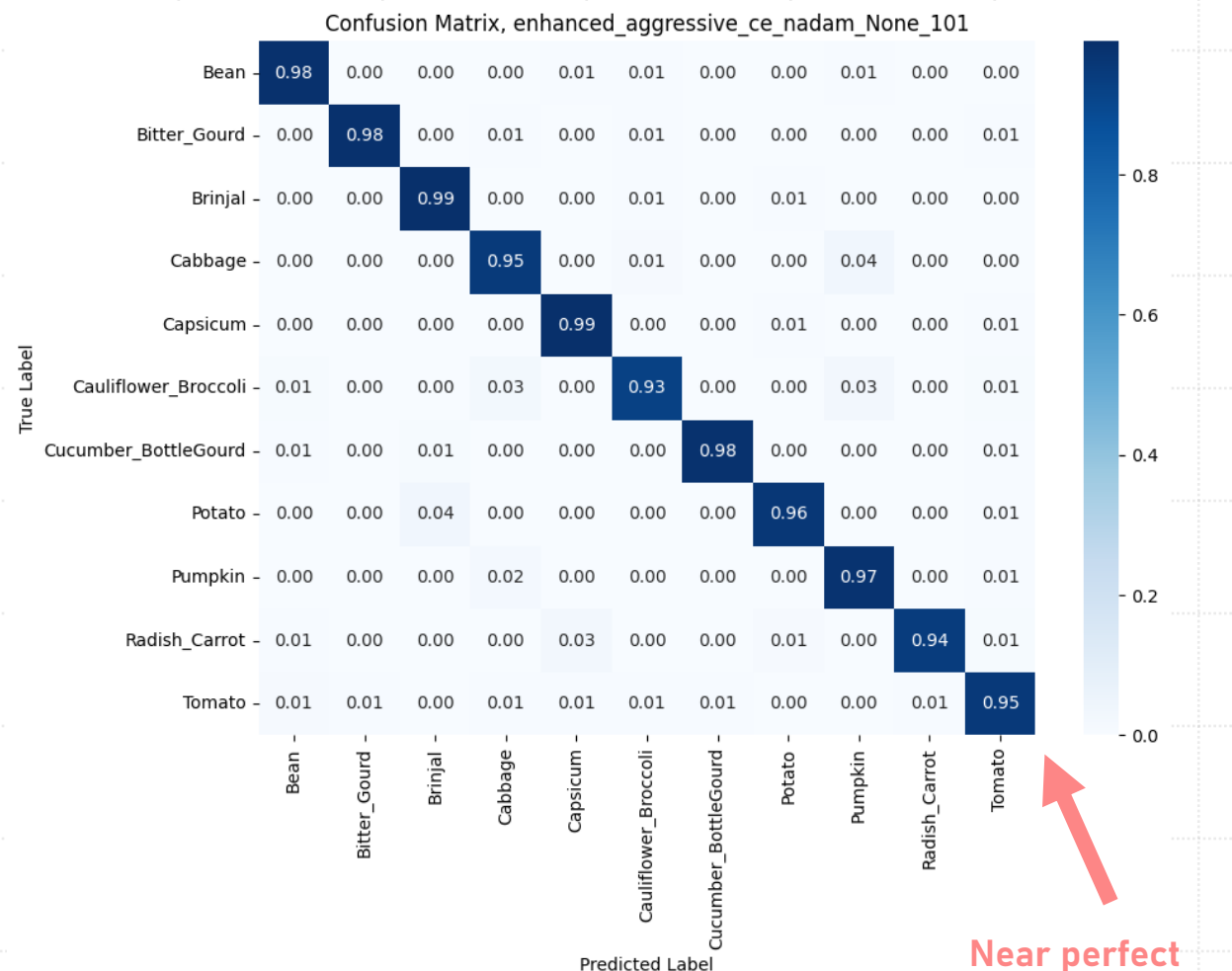
Excellent prediction for each class

Reliability test – Classification report + Normalized Confusion matrix

Best model of 101x101

Classification Report for: enhanced_aggressive_ce_nadam_None_101

	precision	recall	f1-score	support
Bean	0.97	0.98	0.98	200
Bitter_Gourd	0.99	0.98	0.99	200
Brinjal	0.96	0.99	0.97	200
Cabbage	0.95	0.95	0.95	200
Capsicum	0.96	0.99	0.98	200
Cauliflower_Broccoli	0.96	0.93	0.94	200
Cucumber_BottleGourd	0.99	0.98	0.99	200
Potato	0.97	0.96	0.97	200
Pumpkin	0.93	0.97	0.95	200
Radish_Carrot	0.99	0.94	0.97	200
Tomato	0.95	0.95	0.95	200
accuracy			0.97	2200
macro avg	0.97	0.97	0.97	2200
weighted avg	0.97	0.97	0.97	2200



Final Test – Testing best models on custom data (real world test)



Cabbage



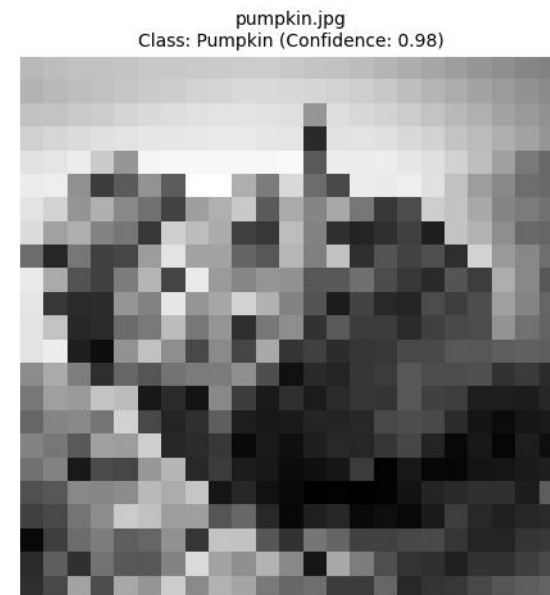
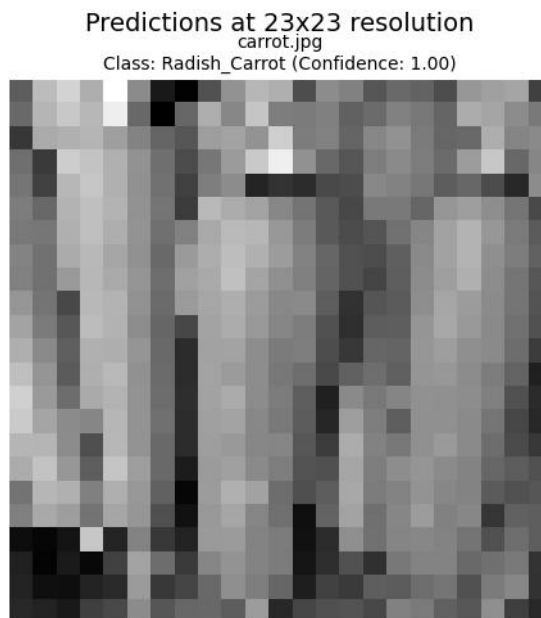
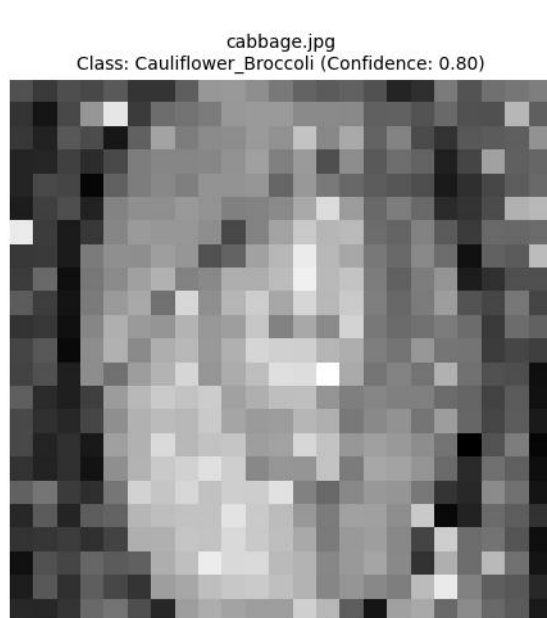
Pumpkin



Carrots

Final Test – Testing best models on custom data (real world test)

Best 23x23 Model Prediction:



Confused with broccoli due to similarities
Confidently correct on others

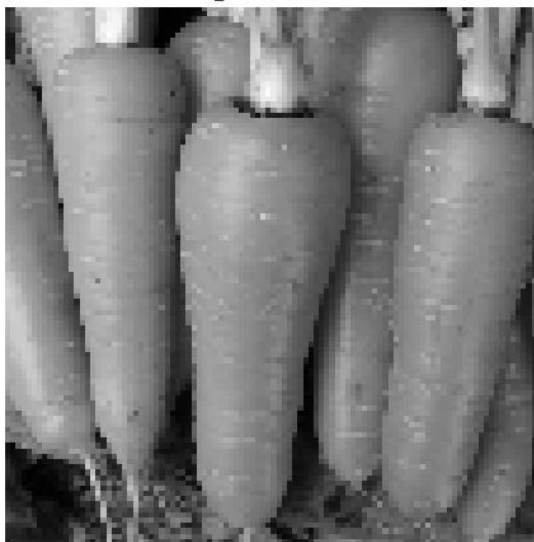
Final Test – Testing best models on custom data (real world test)

Best 101x101 Model Prediction:

cabbage.jpg
Class: Cabbage (Confidence: 0.98)



Predictions at 101x101 resolution
carrot.jpg
Class: Radish_Carrot (Confidence: 0.99)



pumpkin.jpg
Class: Pumpkin (Confidence: 0.99)



Confidently correct on all 3, due to higher detail.



End of presentation